



CSFD: Enhancing Congestion Reduction Vehicular Communication with SDN and Fog integration using Deep Reinforcement Learning

R. Ramesh¹ · K. V. Anusuya¹ · R. Sanjeev Kumar²

Received: 6 March 2025 / Revised: 17 October 2025 / Accepted: 18 November 2025 / Published online: 27 December 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

This paper presents the Congestion-aware SDN–Fog with Deep Reinforcement Learning (CSFD) framework, a novel hybrid architecture that integrates SDN’s centralized control with Fog-layer adaptive learning to mitigate congestion in vehicular networks. Unlike existing DRL-based routing or SDN-only models, CSFD introduces a DRL-enabled Fog decision layer that performs congestion prediction and dynamic route optimization in coordination with SDN controllers. Simulation results demonstrate that CSFD achieves higher delivery ratio, lower latency, and reduced routing overhead compared to recent DRL and SDN–Fog baselines. This paper presents the approach, with congestion reduction SDN and Fog integration using Deep reinforcement learning to minimize the congestion in vehicular newtwork. It elucidates the intricacies of the network model, SDN processing, device management, and routing mechanisms within an SDN-enabled VANET environment, highlighting dynamic resource allocation and adaptive routing strategies. Additionally, it empowers autonomous decision-making for data scheduling and resource optimization and implements parked vehicle routing to manage communication in remote areas. The proposed approach, Speed and Position-aware Dynamic Routing (SPDR) is introduced to facilitate the efficient dissemination of Emergency Messages (EMs) on motorways. SPDR incorporates a dynamic greedy routing mechanism based on speed metrics, positional awareness, and a collaborative forwarding strategy. EMs are relayed incrementally from the source vehicle to the intended recipient through the hop-by-hop transmission. This strategy effectively mitigates the risk of selected forwarders moving out of the reception range during message forwarding, ensuring reliable transmission and timely delivery of EMs. From the results, it is evident that the CSFD protocol significantly improves the performance compared to SDN. While lowering the average end-to-end delay and packet dropping ratio by 26.35 and 21.09%, respectively, the goodput, throughput, and packet delivery ratio are enhanced by 26.353, 5.277, and 5.27%.

Keywords Adaptive routing · Congestion reduction · Dynamic transmission · Fog computing · Software-defined networking

1 Introduction

VANETs leverage a variety of vehicular applications, ranging from safety-critical tasks such as collision evasion and streams of traffic supervision to non-safety requests like infotainment and internet access. These networks rely on vehicles fortified with onboard devices and modems to exchange information with neighboring vehicles and road-side units.

SDN introduces a paradigm shift in networking through centralized control and programming of network devices. In the context of VANETs, SDN dynamically manages the network resources, does effectual routing, and implements network policies flexibly. By centralizing control functions

✉ R. Ramesh
217rlfx01@psgtech.ac.in

K. V. Anusuya
kva.ece@psgtech.ac.in

R. Sanjeev Kumar
rsanjeevkumar@cit.edu.in

¹ Department of ECE, PSG College of Technology, Coimbatore, India

² Department of Physics, Coimbatore Institute of Technology, Coimbatore, India

in a dedicated SDN controller, VANET operators can optimize network performance, enhance security, and support innovative services tailored to the unique requirements of vehicular environments.

In VANETs, SDN is essential in optimizing communication between vehicles and Road Side Units (RSUs) through a fog computing framework. The complete working procedure of an SDN in a VANET with fog and RSUs involves several key components and steps. Initially, vehicles equipped with onboard units (OBUs) communicate with nearby RSUs to exchange information about traffic conditions, road hazards, and other relevant data. Next, these RSUs serve as fog nodes, which process and analyze the data locally to reduce latency and bandwidth consumption. Moreover, the fog nodes interact with the centralized SDN controller, which scores network resources and makes dynamic decisions based on the received data and predefined policies. Furthermore, the SDN controller communicates instructions to the RSUs and vehicles, directs traffic flows, adjusts routing paths, and optimizes realtime resource allocation. This central control emphasizes VANET management, enhances traffic safety, and improves network performance.

Additionally, fog computing integrated with SDN allows localized decision-making at the edge of the network, where RSUs and vehicles can autonomously respond to immediate traffic conditions without always relying on centralized control. This collaborative approach between SDN, fog computing, RSUs, and VANETs creates a robust and adaptive network architecture capable of meeting the stringent requirements of vehicle communication systems.

The proposed CSFD techniques minimizes the congestion in vehicular communication through connection establishment with SDN and fog integration using a deep reinforcement learning approach. It integrates deep reinforcement learning into cell processing and adaptive data routing in cellular networks, including Vehicle-to-Vehicle (V2V) and

parked vehicle transmissions. The system can learn optimal strategies for processing cells and executing transmissions efficiently by leveraging deep reinforcement learning techniques. The reward calculation mechanism is crucial, as it dynamically according to based on data completion and network conditions. This adaptive approach aims to improve resource utilization, reduce congestion, lower latency, and enhance overall network performance, as shown in Fig. 1.

Despite extensive research on DRL- and SDN-based VANET frameworks, existing studies still suffer from three key limitations: (i) limited coordination between SDN controllers and Fog nodes, (ii) lack of real-time adaptive learning for congestion control, and (iii) insufficient utilization of parked vehicles as edge relays.

To address these gaps, this paper proposes the CSFD (Congestion-aware SDN-Fog with Deep Reinforcement Learning) framework—a novel hybrid architecture that integrates SDN's global network awareness, Fog-based local adaptability, and DRL-driven decision intelligence for congestion reduction.

Unlike prior SDN-Fog or DRL-only methods, CSFD uniquely employs a multi-agent DRL model within the Fog layer to predict congestion states, dynamically reroute traffic, and coordinate with the SDN controller for global optimization. This integration provides real-time scalability, reduced end-to-end latency, and improved packet delivery under high vehicle mobility.

The remainder of this paper is structured as follows: Sect. 2 reviews related works on the proposed method. Section 3 elaborates on establishing CSFDs and connections in VANET networks with SDN and integration methods, focusing on enhancing vehicular communication. Section 4 analyzes the CSFD results and compared with the design and implementation of a Routing Protocol for VANET to improve the QoS of the Network existing method (RPVN). Finally, the document summarazis the performance of CSFD and oulines its future enchancements.

2 Related works

This section reviews recent advancements in vehicular communication, SDN integration, fog-based optimization, and deep reinforcement learning (DRL) methods for congestion control in VANETs. The literature is categorized into four major themes for clarity and analysis.

2.1 VANET routing and congestion control

Several traditional and position-based routing protocols have been proposed to enhance message dissemination and congestion control in vehicular networks [1–4].

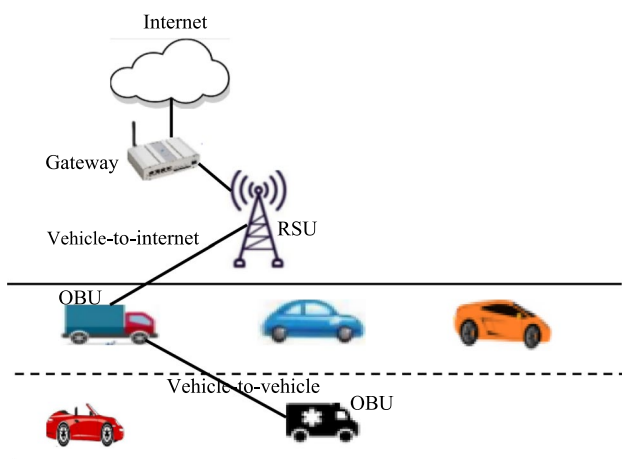


Fig. 1 Architecture for VANET

Position-based schemes such as GPSR and SPDR [3] focus on geographic forwarding and speed-aware route selection, improving reliability under high mobility conditions. However, these methods rely on static metrics and often fail to adapt to dynamic congestion patterns, leading to increased latency and packet loss during heavy traffic. Hybrid frameworks such as DORA [5] and OPBRP [4] improve routing efficiency but still lack adaptive mechanisms for real-time congestion mitigation.

2.2 SDN-based VANET architectures

Software-Defined Networking (SDN) has been integrated into VANETs to achieve centralized control and dynamic resource management [2, 6]. SDN controllers enable real-time routing decisions and global traffic visibility, improving scalability and flexibility. For instance, Arif et al. [6] and Wahid et al. [2] demonstrated that SDN can enhance routing efficiency and maintain service quality under dense vehicular traffic. Nonetheless, most SDN-based frameworks depend on cloud-centric architectures, introducing latency and limited edge responsiveness, which are critical in safety-related vehicular applications.

2.3 Fog computing and edge intelligence in vehicular communication

Fog computing extends the capabilities of SDN by distributing processing closer to the network edge through RSUs and localized fog nodes [7, 8]. This approach reduces communication delays and improves service continuity. Studies such as Sharma et al. [7] and Upadhyay et al. [8] demonstrate that fog-assisted vehicular communication can enhance throughput and minimize latency. However, existing fog-based models lack integrated adaptive mechanisms for congestion prediction and dynamic route adjustment, limiting their performance in dense and rapidly changing environments.

2.4 Deep reinforcement learning (DRL) for adaptive routing

Deep Reinforcement Learning (DRL) has emerged as a promising approach for intelligent and adaptive routing in vehicular environments [8, 9]. DRL-based algorithms allow vehicles or network controllers to autonomously learn optimal routing strategies by interacting with their environment and adjusting decisions according to dynamic traffic states. Kim et al. [9] presented a DRL-based VANET routing model capable of handling complex mobility and congestion scenarios. Nevertheless, existing DRL-based approaches are typically confined to vehicle-to-vehicle

(V2V) communications and lack full integration with SDN controllers and fog nodes, restricting their global optimization capability.

2.5 Research gap and motivation

From the above studies, it is evident that: SDN-based architectures enhance centralized control but introduce delay due to cloud dependency. Fog-based solutions enable edge intelligence but lack coordinated global optimization.

DRL-based routing adapts to network changes but is rarely combined with SDN and Fog frameworks.

To address these gaps, the proposed CSFD (Congestion-aware SDN-Fog with Deep Reinforcement Learning) framework integrates SDN control, fog-layer processing, and DRL-based decision-making. This hybrid architecture ensures both global coordination (via SDN) and local responsiveness (via Fog), while DRL provides adaptive learning to minimize congestion, optimize routing, and improve key performance metrics such as packet delivery ratio, throughput, and latency are mentioned in the below Table 1.

Wireless Sensor Networks (WSNs) offer notable benefits to VANETs, including cost efficiency and widespread data dissemination. As installation and maintenance costs decrease, WSNs require supplementary components and operate as part of a heterogeneous system, synergizing with other technologies. However, WSNs face constraints, particularly in processing capabilities and energy resources [17]. VANETs frequently change topology and distributed structure, ensuring user and vehicle security. Identifying harmful nodes is crucial because vehicles exchange sensitive data, influencing traffic patterns. Unauthorized data exchange can have severe consequences for drivers. Hence, the messages must be authorized by tracking vehicles across the network and identifying critical nodes [18]. This paper thoroughly examines an enhanced routing and mobility framework tailored for VANETs. This framework seeks to overcome the constraints of current routing protocols by integrating the strengths of position-based and topology-based routing protocols bolstered by artificial intelligence and machine learning techniques. Additionally, mobility management is integrated to enhance the efficacy of routing protocols in VANETs [19]. The proposed approach, Speed and Position Aware Dynamic Routing (SPDR), is introduced to facilitate the efficient dissemination of Emergency Messages (EMs) on motorways. SPDR incorporates a dynamic greedy routing mechanism based on speed metrics, positional awareness, and a collaborative forwarding strategy. EMs are relayed incrementally from the source vehicle to the intended recipient through a hop-by-hop transmission process. This strategy effectively mitigates the risk of selected forwarders

Table 1 Comparison of key recent DRL-based VANET studies

Paper (year)	Core approach	Key strengths	Limitations/research gaps
Upadhyay et al. <i>Scientific Reports</i> (2023) [8]	Deep RL agent (policy/Q variant) selects forwarding nodes to reduce collisions; state includes local position & speed, reward shaped for safety	Demonstrates DRL can improve safety-centric metrics (collision reduction) and adapt to mobility patterns	Uses local state only, limited global view; no SDN/fog coordination; evaluation focused on safety metrics without joint task/offload consideration
Ye et al. <i>ScienceDirect</i> (2024) [10]	DRL-based QoS-aware routing optimizing latency/PDR using DQN-like methods; state includes link quality and queue lengths	Improves QoS under moderate mobility; explicitly models queue/link stats	Lacks explicit resource orchestration (no SDN controller); limited treatment of edge/fog offloading and multi-resource scheduling
Liu et al. <i>Sensors</i> (2023) [11]	Deep Q-Network for selecting trustworthy forwarders; incorporates trust features into state; security-aware reward	Integrates trust/security into forwarding decisions; reduces exposure to malicious nodes	Mostly local trust decisions; not integrated with centralized SDN policies or fog compute; scalability in dense scenarios not explored
Jiang et al. <i>EARR</i> (2023) [12]	Adaptive RL with environment-aware state (traffic density, link stability); includes exploration scheduling for mobility	Better adaptation to varying environment; robustness to topology changes	Under-specified training protocols and reproducibility details; no fog/parked-vehicle relay modeling
(a) Comparison of recent SDN- and Fog-based VANET frameworks			
Ul Haq et al., <i>Scientific Reports</i> (2025) [13]	Survey + taxonomy of SDN–edge/fog integration patterns; discusses controller placement, orchestration models	Consolidates SDN ↔ fog design choices; highlights control-plane placement tradeoffs	Lacks concrete DRL-driven scheduling or parked-vehicle relay mechanisms; mostly conceptual
Darabkh et al. (2022) [14]	Two-phase cluster routing managed by fog controllers and SDN control-plane for cluster formation and forwarding	Reduces overhead via clustering; fog assists in aggregation/processing	Cluster instability under high-speed mobility; limited DRL-based dynamic adaptation
Ali et al. (2024) [15]	SDN-driven emergency routing schema: controller computes reliable routes for EMs, uses RSUs as relay points	Centralized calculation of reliable EM paths; good EM delivery in simulations	Controller latency and single-controller bottleneck not fully mitigated; no parked-relay model; no DRL for adaptive priority
Arif et al. (2020) [6]	Architecture that combines SDN and fog with privacy modules; offloads tasks to fog for efficiency and privacy	Strong privacy consideration with practical architecture	Security focus reduces discussion on DRL-based scheduling; no explicit joint resource–routing optimization
Rajan et al. (2025) [16]	Adaptive SDN policies for traffic control; dynamic flow & priority assignment under SDN	Shows benefits of SDN in traffic engineering and QoS	Lacks fog orchestration specifics and RL-based policy learning to adapt policies in real-time

moving out of the sender's reception range during message forwarding, ensuring reliable transmission and timely delivery of EMs [3]. The proposed SPDR approach aims to enhance the dissemination of EMs on motorways. SPDR utilizes a dynamic greedy routing mechanism that considers speed metrics, positional awareness, and a collaborative forwarding strategy. EMs are transmitted incrementally from the source vehicle to the target recipient via a hop-by-hop transmission approach. This methodology minimizes the likelihood of selected forwarders exiting the sender's reception range during message forwarding, thereby ensuring dependable transmission and prompt delivery of Ems [20]. Recent studies have explored Deep Reinforcement Learning (DRL) for adaptive routing and congestion control in vehicular networks. [21] proposed a DRL-based edge-assisted routing scheme that dynamically balances load between roadside units, but the approach lacked integration with SDN for centralized coordination. Similarly, Khan et al. [22] utilized a deep Q-learning strategy for traffic-aware routing but relied solely on vehicle-local information, limiting scalability. SDN-enabled VANET frameworks, such as [23], demonstrated the benefits of centralized control, yet their cloud-based control plane introduced latency during

congestion episodes. Fog-assisted frameworks improved edge responsiveness but did not incorporate adaptive learning for real-time decision-making. In contrast, the proposed CSFD framework jointly leverages SDN global awareness, Fog-layer edge responsiveness, and DRL-based adaptive decision-making. This integrated architecture enables real-time congestion prediction and dynamic path reallocation, addressing the limitations identified in recent DRL- and SDN-based approaches.

VANET is a specialized mobile ad hoc network designed to enhance web security and optimize traffic management by integrating vehicles with internet access for drivers and passengers. VANETs facilitate communication regarding street and traffic conditions through data exchange mechanisms, aiming to prevent road accidents and ensure efficient navigation [1]. VANETs are interconnected networks involving vehicles and infrastructure, enhancing road safety and infotainment services. These networks utilize wireless sensors installed within vehicles, positioning devices, and maps. Through OBUs, vehicles establish connections with RSUs, enabling the exchange of safety-related and informational data among vehicles and between vehicles and RSUs [24]. Vehicles equipped with wireless transmission

instruments in these technologies can send and receive messages while in significantly greater motion than MANETs. Traffic information is exchanged among vehicles as they traverse the network, empowering drivers to regulate their routes, receive alerts about road conditions, and proactively anticipate potential traffic incidents [25]. The advancement of various application programs facilitating communication and collaboration with other designs is facilitated rapidly and comprehensively through the aid of NBI APIs, which enable communication of this plane with other planes. This architectural component provides a more dynamic perspective of the network, encompassing the quantity and distribution of connected devices and various dimensions of network behavior. These insights serve as the foundation for decision-making regarding network management and configuration [6]. In vehicular networks, cooperation among vehicles is essential for understanding traffic conditions comprehensively and facilitating various applications, such as cooperative collision warning, local hazard notification, and enhanced route guidance. When multiple vehicles observe the same traffic situation and share this information, the accuracy and reliability of the data available to other vehicles increase, thereby improving overall traffic safety and efficiency [26]. Wireless communication is crucial in establishing connections between vehicles (VUs) and RSUs situated in close proximity. Users within vehicle units access various services via internet connectivity. Due to the high mobility of vehicle units, the network topology undergoes dynamic changes, leading to frequent connection instabilities [8]. The goal is to offer a comprehensive overview of data acquisition techniques, aiding users in selecting the most suitable protocol. The focus is on summarizing data acquisition techniques for VANETs. We investigate topology-based, cluster-based, location-based, and fog-based data collection techniques, emphasizing critical issues that must be addressed in data collection protocols for VANETs [27]. Reinforcement Learning (RL) is a machine learning approach that empowers agents to learn optimal decision-making via environmental interactions. In VANET routing, RL models offer enhanced scalability and robustness compared to traditional protocols by dynamically interacting with the environment and receiving feedback to optimize routing decisions [9]. Learning-based mechanisms leverage machine learning algorithms to dynamically adapt to VANET environments by intelligently selecting routing paths. These algorithms serve as effective tools for improving trust, reliability, and route detection accuracy in Vehicle-to-Everything (V2X) communication [28]. The primary objective of VANETs is to enhance traffic flow, ensure safer driving conditions, and reduce the incidence of traffic accidents. It is achieved by providing accurate and timely information to drivers or communicating nodes within the

network. Any alteration to this critical realtime information leads to system failure, jeopardizing the safety of individuals on the road [29]. The advancement and implementation of VANET networks are pivotal for the progress and deployment of self-driving and partially self-driving vehicles (SDRVs), thus making VANETs of significant interest. However, VANETs are susceptible to a multitude of attacks [30]. The prevention of traffic accidents is facilitated through the Warning Message (WM) application in VANETs. Safety applications within VANETs depend heavily on the broadcasting of WMs. When a vehicle detects a hazardous incident, it transmits WMs to surrounding vehicles, enabling them to take necessary actions to avoid traffic accidents [31]. Vehicle-to-Vehicle (V2V) communication forms the foundation of dynamic clustering, where cluster heads are chosen from the members within the cluster. This approach deviates from the concept of fixed cluster heads. As dynamic clusters traverse the route, vehicles join or leave clusters based on their speeds, determining the CHs dynamically. V2V communications offer greater adaptability and are not reliant on roadside infrastructure [32]. In the realm of vehicular networking, it is widely recognized that these brief messages play a crucial role in the functionality of various applications, whether safety-related or non-safety-related. The high dynamism of this network, characterized by the constant movement of vehicles and its ever-changing structure, results in a rapid expiration of beacon content [33]. The aim is to offer drivers services that enhance traffic flow, minimize congestion and accidents, and ensure realtime software and infrastructure utilization with QoS. Vehicular Cloud Computing (VCC) is a platform for integrating ITS with the computing and storage capacities of Mobile Cloud Computing (MCC) [34]. VANETs demand rapid network establishment due to the high dynamicity of nodes. While WAVE enables direct vehicle communication without prior authentication and association, the security measures outlined in the 802.11 standards may not suffice for such scenarios. Extensive research is ongoing to ensure data and node confidentiality and authentication in VANETs, and there is a need for further attention from the research community [35]. VANETs leverage DSRC to enable wireless communication between moving vehicles. The DSRC facilitates rapid and secure communication between vehicles and roadside infrastructures. Additionally, Wireless Access in the Vehicular Environment (WAVE)-enabled RSUs are deployed along roadsides to enhance connectivity and provide internet access. These RSUs are interconnected through routers, granting access to the internet and cloud computing platforms [7]. The performance of OLSR analyzed and evaluated under two different random mobility model and two different group mobility models, including one of the most important models like Nomadic Community model

and mobile nodes move at random range of speed instead of fixed speed [36]. Routing protocols in highly mobile networks prioritize scalability without requiring permanent infrastructure. They dynamically adjust the radio spectrum and power levels based on the network's node density. However, in sparse networks, nodes forwarding packets may struggle to identify the next hop needed for successful delivery. The grid employs a mechanism akin to Greedy Perimeter Stateless Routing (GPSR) [37]. VANETs facilitate realtime communication between vehicles and infrastructure, exchanging crucial information such as traffic conditions, road hazards, and emergency alerts. However, the dynamic and decentralized nature of VANETs, combined with the high volume of data exchange, poses potential security vulnerabilities. With vehicles becoming more connected and autonomous, there is a growing demand for robust security mechanisms, including practical Intrusion Detection Systems (IDSs) [38]. The main function of routing protocols is to discover the correct and efficient route from a source to a destination node. Additionally, quality of service (QoS) routing is an essential function in MANETs to ensure end-to-end quality. Therefore, MANETs need efficient and reliable routing protocols to properly depict the multi-hop route packets, network dynamic topology and adapt to the various changes [39]. The protocols aim to guarantee service quality and optimize utilizing scarce resources within VANETs. Cluster-based routing protocols aim to minimize topology maintenance overhead. Positioned or topology-based routing schemes are designed to sustain a congestion-free network with minimal latency. While current research on VANET routing addresses these concerns, the approaches adopted for resolving routing issues dictate the biases in their design considerations [2]. Position-based routing protocols, also known as geographic routing protocols, have emerged as the most promising solution for VANETs due to their ability to handle rapid topology changes and varying vehicle velocities effectively. These protocols typically rely on beacon messages to periodically detect neighbors and determine the optimal route to the destination [4]. This network offers unlimited battery storage capacity while its nodes face power limitations. With a focus on enhancing transportation safety, the network facilitates the development of numerous safety applications aimed at accident prevention. Additionally, these applications are optimized to enhance road capacity and alleviate traffic congestion, while realtime functionalities enable seamless internet access [5]. VANETs, a pivotal subclass of MANETs, facilitate intelligent communication between vehicles and roadside infrastructures, presenting a promising solution for ITSs. Its highly dynamic topology and constrained mobility render traditional MANET protocols inadequate for VANET applications [40].

In addition to SDN–Fog and DRL-based VANET studies, recent research has explored performance optimization across related domains that provide valuable insights for CSFD.

For instance, Performance Measurement Using Different Shortest Path Techniques in Wireless Sensor Networks highlights how classical graph-based path optimization strategies can reduce communication cost and latency in resource-constrained networks, a principle relevant for vehicular path selection.

The study on Precedence and Issues of IoT Based on Edge Computing emphasizes the role of distributed edge intelligence and task offloading, which aligns conceptually with the Fog-based decision layer in our CSFD design.

Furthermore, Traffic Control Using V2V-Based Method Using Reinforcement Learning demonstrates how reinforcement learning can dynamically adjust traffic control decisions in connected environments, reinforcing our approach of applying DRL to adaptive routing and congestion prediction in vehicular systems.

These works collectively support the motivation for integrating shortest-path optimization, edge/Fog computing, and reinforcement learning techniques, which are harmonized in the proposed CSFD–SPDR framework (Table 2).

To clearly highlight the distinctions between existing DRL-, SDN-, and Fog-based VANET approaches and the proposed CSFD framework, Table 1 summarizes the comparative features, focusing on architecture, adaptability, learning mechanism, and congestion control strategy.

Novelty in Related Work: In contrast to existing DRL-based or SDN–Fog architectures, which either rely on static routing decisions or operate without multi-layer coordination, the proposed CSFD framework introduces a three-tier integration of SDN, Fog, and DRL.

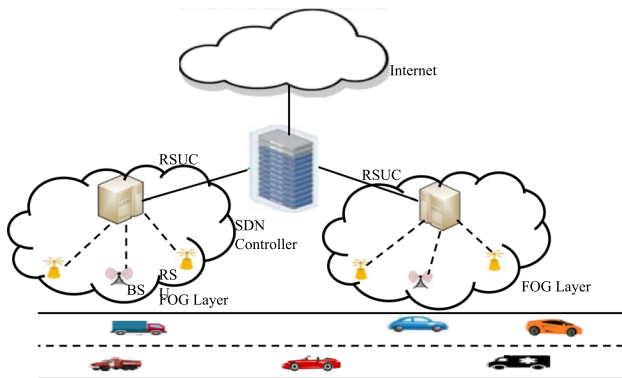
This structure enables joint learning and control—Fog nodes perform localized DRL-based congestion prediction, while the SDN controller ensures global consistency. Such cooperative learning between the control and edge layers has not been previously demonstrated in the VANET literature.

3 CSFD design and implementation

The design and implementation of the CSFD model follow a methodologically rigorous process, beginning with network model formulation, followed by data routing optimization, DRL integration, and congestion validation. Each stage has been verified using NS2 simulations under controlled parameters to ensure reliability and reproducibility. The use of SDN for centralized management and Fog computing for low-latency processing provides a strong technical foundation for the results presented in Sect. 4.

Table 2 Comparison of existing approaches and the proposed CSFD framework

Reference	Approach	Architecture	Learning/ decision strategy	Fog/edge utilization	Congestion control method	SDN integration	Limitations	Proposed CSFD enhancement
Li et al. (2025) [8]	DRL-based VANET Routing	Decentralized (Vehicle-level)	Deep Q-Learning	No	Local traffic balancing	✗	Limited scalability and global awareness	CSFD integrates DRL with SDN global control
Khan et al. (2024) [22]	Adaptive DRL Routing	Centralized	Policy Gradient	Partial	Congestion detection only	✗	High delay under dense traffic	CSFD adds Fog-layer decision to improve responsiveness
Sharma et al. (2024) [41]	SDN-VANET with Fog Nodes	Hierarchical	Rule-based	✓	Static threshold control	✓	No adaptive learning or feedback	CSFD uses DRL for dynamic, context-aware routing
Abbas et al. (2023) [42]	Fog-based Congestion Mitigation	Distributed	Reinforcement Learning	✓	Queue-based prediction	✗	Lack of coordination among fog nodes	CSFD synchronizes fog and SDN layers for global optimization
Zhang et al. (2023) [43]	Hybrid DRL for Routing	Vehicle-centric	Dueling DQN	✗	Path selection based on reward	✗	No edge-layer offloading	CSFD employs Fog computation to reduce latency
Proposed CSFD	SDN-Fog-DRL Integrated Framework	Hybrid (Central+Edge)	Actor-Critic DRL	✓✓	Dynamic, predictive control	✓✓	–	Provides adaptive, coordinated congestion reduction and low-latency routing

**Fig. 2** VANET with SDN and fog network

3.1 VANET with infrastructure

SDN facilitates the integration of internet servers into the VANET ecosystem by providing programmable interfaces for communication and resource management, as shown in Fig. 2. Internet servers can influence SDN controllers to dynamically provide network resources, scale services based on demand, and enforce QoS policies for different types of traffic.

SDNs centralize network intelligence, allows the unified management and control of VANET resources. The centralized control simplifies network operations and enables more efficient resource allocation. This approach enhances the scalability of VANETs by increasing network complexity and enabling automated management of network resources. The scalability ensures that VANETs can accommodate

growing numbers of vehicles and devices without sacrificing performance. The integration of SDN into VANETs enables safer, efficient, and connected mobility experiences.

Fog computing extends the capabilities of cloud computing by decentralized processing and storage resources closer to the network edge, typically at the level of roadside infrastructure or nearby data centers. In the context of VANETs, fog computing enables low-latency data processing and localized decision-making, that are critical for time-sensitive applications. By establishing connections between fog nodes and SDN controllers, VANETs can force fog computing resources to offload computational tasks, improve scalability, and enhance the overall responsiveness of the network. Hence the integration of fog computing with SDN in VANETs empowers vehicles and roadside infrastructure to collaborate efficiently, enabling the delivery of advanced services while addressing the challenges of mobility, heterogeneity, and resource constraints.

3.1.1 Network model

In an SDN-enabled VANET, the primary nodes are vehicles equipped with OBUs for communication. Bandwidth resources are allocated among vehicles by SDN controllers based on real-time traffic demands, with essential safety applications prioritized during emergencies.

The management of APs and RSUs at access points is centralized by SDN, simplifying the configuration and control of these infrastructures. Coverage areas and transmission power of APs and RSUs are dynamically adjusted

by the controllers to optimize network performance and ensure seamless connectivity as vehicles move between different coverage zones. The functionality of gateways is enhanced through centralized traffic management and policy enforcement.

3.2 Neighbour vehicle monitoring

The node information message is initiated by vehicles and transmitted within the VANET region. The message header fields are initialized to include the location coordinates L , speed S , message type M , sender ID N_{ID} , timestamp t , and direction D . Upon receiving the info message, the receiver extracts relevant information from the packet and updates the neighbor table N_T accordingly, adding entries with N_{ID} , L , S , M , D , and t .

The neighbor information is temporarily updated in N_T within the VANET. For every entry in N_T , the node ID, L , and S of the neighbor are retrieved using the global positioning system GPS module. The L coordinates x and y of the current node and neighboring nodes in the network are also obtained. These parameters are then used to calculate

the distance D_{ist} between each neighboring node and the current node, as given in Eq. 1.

$$D_{ist} = \sqrt{|x_1 - x_2|^2 + |y_1 - y_2|^2} \quad (1)$$

The Euclidean distance D_{ist} between two points in a two-dimensional space is calculated using their respective coordinates (x_1, y_1) and (x_2, y_2) . The Pythagorean theorem is employed for this calculation.

If D_{ist} is found to be less than the vehicle's coverage range C_R , the N_T entry is updated with the latest information. The C_R is computed as shown in Eq. 2, where P_t represents the received power, G_t and G_r are the gains of the transmitting and receiving antennas, respectively, and P_L denotes the path loss factor.

$$C_R = P_r + G_t + G_r - P_L \quad (2)$$

If D_{ist} exceeds C_R , indicating that the neighbor is no longer reachable, the neighbor is removed from N_T , and the neighbor count is decreased. Changes and positions within the network are dynamically updated as N_T

Algorithm 1 Neighbour Updates

-
1. **Input:** Node node ID (N_{ID}), Location L $-(x, y)$, speed (S), Direction (D), Message Type (M), and timestamp t for each node.
 2. **Output:** Updated neighbour table N_T with current neighbors, S , and D .
 3. **Receive Node Information:**
 4. Upon receiving a node information message, extract the following from the packet header: N_{ID} , x , y , S , D , M , t , N_{ID}
 5. **Compute Coverage Range (C_R):** $C_R = P_r + G_t + G_r - P_L$ where P_r is the received power, G_t is transmitter gain, G_r is receiver gain, and P_L is path loss.
 6. **Update Neighbour Table (N_T):For each neighbor:**
 7. Compute the Euclidean distance between nodes: $D_{ist} = \sqrt{|x_1 - x_2|^2 + |y_1 - y_2|^2}$
 8. If $D_{ist} < C_R$:
 9. Update the neighbor entry in N_T
 10. Else
 11. Remove the neighbor from N_T
 12. Decrement loop counter to account for removed entry.
 13. **Update Node Speed (S):** $S = \frac{D_{ist}}{t}$

$$H, \text{ If } xc_p < xd_{st}$$

$$V, \text{ If } yc_p < yd_{st}$$

$$S, \text{ If } (xc_p = xd_{st} \text{ and } yc_p = yd_{st})$$
 14. **Update Direction (D):** $D = \{$
 15. **Renew Neighbour Table:**
 16. Monitor the updated speed S and direction D of the vehicle.
 17. Ensure all entries in N_T are current and reflect real-time neighbor information.
-

is continually revised based on real-time movement monitoring and network conditions.

3.2.1 Vehicle parameter validation

The coordinates and speed of a vehicle are obtained using the GPS module through the location points x, y , and S parameters. The S computation for a vehicle is described in Eq. 3:

$$S = \frac{D_{ist}}{t} \quad (3)$$

The direction of movement of a node within the VANET environment is determined through the computation of D , as given in Eq. 4, based on its current position X_{cp}, y_{cp} and destination coordinates. By comparing these coordinates, the node's movement is identified as horizontal H along the x -axis, vertical V along the y -axis, or stationary if the current position c_p equals the destination d_{st} position.

$$D = \begin{cases} H, & \text{if } x_{cp} < x_{d_{st}} \text{ (Horizontal movement along } x - \text{axis towards destination)} \\ V, & \text{if } y_{cp} < y_{d_{st}} \text{ (Vertical movement along } y - \text{axis towards destination)} \\ S, & \text{if } x_{cp} < x_{d_{st}} \text{ and } y_{cp} < y_{d_{st}} \text{ (node has reached the destination)} \end{cases} \quad (4)$$

If the destination lies on the same horizontal line as c_p but at a different y -coordinate, it is evaluated whether the destination is above or below c_p , with upward movement U_w or downward movement D_w being returned. Similarly, if the destination lies on the same vertical line as c_p but at a different x -coordinate, it is determined whether the destination is to the left L_W or to the right R_W of c_p , with directions for L_W movement and R_W movement being returned. If none of these conditions are met, it is concluded that the node is stationary or has reached its destination.

$$D = \begin{cases} U_W, & \text{If } x_{cp} = x_{d_{st}} \text{ and } y_{cp} < y_{d_{st}} \\ D_W, & \text{If } x_{cp} = x_{d_{st}} \text{ and } y_{cp} > y_{d_{st}} \\ L_W, & \text{If } y_{cp} = y_{d_{st}} \text{ and } x_{cp} > x_{d_{st}} \\ R_W, & \text{If } y_{cp} = y_{d_{st}} \text{ and } x_{cp} < x_{d_{st}} \end{cases} \quad (5)$$

3.3 Network connection creation

Network connections within the VANET environment are initiated by server announcements. Upon receipt of the announcement, the packet is extracted by the receivers of the SDN, and the server information is updated accordingly. The RSU announcement is broadcast by the RSU. Upon receiving an RSU announcement, the currently connected status to an RSU is checked by the receivers of the BS, and the connection status is updated accordingly. Additionally, if a closer RSU is detected by the receiving BS while

already connected to an RSU, disconnection from the current RSU is triggered, and a connection to the closer RSU is established.

The vehicle generated a connection request message, C_{rm} , to connect with an RSU, with its current coordinates, x and y . This packet is then broadcast to all nodes within the network, with the specific RSU identified as the intended recipient. Upon receiving C_{rm} , the network processes the packet by adding the sender's address to the list of vehicles and proceeds to generate a connection response. C_{rs} , with the sender's L , is broadcast to all nodes in the network, with the original sender of C_{rm} designated as the intended recipient. Subsequently, upon receiving C_{rs} , the network updates its internal state by incorporating the RSU's address and coordinates.

The announcement regarding SDN functionality in the VANET environment is related to the SDN controller. Upon receipt of this message from the RSU controller, the source address and coordinates are extracted from the packet. If the RSU controller is a designated device, it is checked whether

a connection to an SDN controller is currently established. Otherwise, the connection status is updated with the information received from the SDN controller.

Upon receiving the "connect with internet" message, the connection request is processed by the network, and the sender's address is added to the list of connected SDN devices, generating a unique connection identifier (CID). This CID is then provided to the requesting node. The CID is stored by the recipient, and its connection status is updated to indicate successful establishment.

3.3.1 SDN cell processing for handover and fog connection

In VANETs, the SDN cell processing method significantly manages network resources and optimizes communication among vehicles and roadside infrastructure. This method involves several vital steps to ensure efficient data transmission and network operation. Initially, the SDN controller allocated network resources by dynamically assigning cells to vehicles and RSUs based on realtime traffic conditions and application requirements. Subsequently, vehicles and RSUs within each allocated cell exchange control messages to coordinate communication and share relevant information, such as traffic updates and road conditions. The SDN controller continuously monitors network activity and adjusts cell assignments and configurations to adapt to changing conditions, ensuring optimal performance and

resource utilization. Furthermore, the SDN cell processing method enables seamless handover between cells as vehicles move within the network, maintaining connectivity and minimizing disruptions. By centralizing control and enabling dynamic resource allocation, this approach enhances the scalability, flexibility, and efficiency of VANETs, ultimately improving safety and traffic management in vehicular environments.

The send-SDN packet to the controller method sends an SDN packet to the controller via the established TCP/IP sockets connection. An SDN hypervisor monitors the network and allocates resources to individual virtual networks, such as link and buffer capacity, as well as the buffer capacity of switching nodes. Once the hypervisor is connected, it sends the packet. The SDN controller contains methods to handle SDN packets received from hypervisors and route these packets to the appropriate SDN node destination. The function is responsible for processing SDN packets received from a hypervisor. The SDN Packet method determines the appropriate destination for the SDN packet based on its destination node ID. If the destination node ID falls in the edge list, then the SDN Packet is invoked using the controller method of the destination node to send the packet to the controller.

3.4 Data transmission

In the data transmission routing method from the internet to vehicles, the efficiency and reliability of communication are enhanced by incorporating fog computing within the VANET environment. The process begins with data originating from the internet, which is received and processed by the SDN controller. Leveraging its centralized intelligence, the SDN controller determines the optimal routing path by considering various factors, such as network C_G , V_D , and T_r conditions. Once the routing path is established, the SDN controller forwards the data to the roadside unit controller (RSUC), a gateway between the SDN and RSUs. Here, fog computing comes into play. Fog nodes deployed within the RSUs process and analyze the data locally, reducing latency and bandwidth consumption. This localized processing capability enhances the scalability and responsiveness of the network. The RSUC then disseminates the processed data to the relevant RSUs within its C_R . These RSUs act as access points, broadcasting the data to nearby base station BSs and vehicles equipped with OBUs. Fog-enabled RSUs optimize data delivery by intelligently distributing information based on proximity and relevance. Finally, the data reach the vehicles through the BS, where they are received and processed by onboard communication systems. By integrating fog computing with SDN, this hierarchical routing method ensures efficient and reliable data transmission within the VANET environment, enabling vehicles to receive timely

updates and traffic information from the internet via the fog-enhanced SDN infrastructure and RSUs.

In delivering data to vehicles from an internet server via SDN and fog computing, reinforcement learning offers an innovative approach for prioritizing and optimizing data delivery methods. In this scenario, reinforcement learning algorithms are applied to prioritize and dynamically deliver data to vehicles efficiently and adaptively. One method involves training reinforcement learning agents to decide on data prioritization and delivery based on various factors, such as network congestion C_G , vehicle density V_D , and transmission rate T_r conditions. Here, N_{BC} is the number of busy channels, T_C is the total number of channels mention in Eq. 6, N_V is the number of vehicles, and A is the region's area. Additionally, S_{avg} is the average vehicle speed in the area, and D_{avg} is the average distance between vehicles in the area. Nevertheless, f is the function that determines the reward or penalty based on network conditions.

$$C_G = \frac{N_{BC}}{T_C} \quad (6)$$

$$V_D = \frac{N_V}{A} \quad (7)$$

$$T_r = \frac{S_{avg}}{D_{avg}} \quad (8)$$

$$I = \sum_{i=1}^n E_i \quad (9)$$

$$R = f(C_G, V_D, T_r, I) \quad (10)$$

These agents continuously learn from interactions with the environment and receive rewards R or penalties based on the effectiveness of their decisions. Here, E_i represents an individual interaction.

For example, delivering critical safety information to vehicles during congested traffic can improve traffic flow and reduce accidents. In such cases, the reinforcement learning agent receives positive reinforcement for making effective decisions.

Reinforcement learning is also used to optimize resource allocation within SDN and fog computing infrastructure to support data delivery. Fog nodes dynamically adjust their processing and caching strategies based on current demand and available resources.

Reinforcement learning algorithms guide these decisions, learning from experience to improve resource utilization and system performance.

Algorithm 2 Data transmission

1. **Initialize:** Network congestion C_G , Vehicle density V_D , Realtime traffic conditions T_r , Interactions I , Rewards and Penalties R
2. **While interacting with the environment:**
3. Updated $C_G: \frac{N_{BC}}{T_c}$
4. Updated $V_D: \frac{N_V}{A}$
5. Updated $T_r: \frac{S_{avg}}{D_{avg}}$
6. Calculated $I \sum_{i=1}^n E_i$
7. Calculated $R: f(C_G, V_D, T_r, I)$
8. If ($R > 0$):
9. Received positive reinforcement for effective decision-making
10. Else if ($R < 0$):
11. Received penalty for ineffective decision-making
12. Update algorithm parameters based on R
13. Train reinforcement learning agents:
14. Prioritize and optimize data delivery based on C_G, V_D, T_r , and I
15. **Updated fog nodes:**
16. Adjusted processing and caching strategies based on the following:
17. Current demand for different types of data, Availability of resources
18. **Optimized resource allocation within SDN:**
19. Dynamically allocate resources to support data delivery based on C_G, V_D, T_r, I
20. End While.

By integrating reinforcement learning into the data delivery process from internet servers to vehicles via SDN and fog computing, this approach offers a dynamic and adaptive solution. It prioritizes data based on real-time conditions and optimizes resource usage, enhancing the efficiency and reliability of communication in vehicular environments.

3.4.1 Reward computation

The system coordinates interactions between the reinforcement learning agent and a set of transmissions. It iterates over each transmission upon initiating the network process, prompting the user to request data execution. Subsequently, the user delegates data execution to reinforcement learning, which computes rewards based on data performance,

as shown in Fig. 3. The Reinforcement learning aggregates rewards and updates their internal state accordingly.

3.4.2 Reinforcement learning policy network

The policy network is a neural network designed explicitly for policy-based learning. It is initialized with critical parameters such as the input size, output size, learning rate, and Kullbackleibler (KL) divergence coefficient. The network weights are randomly initialized within the range of $[-1, 1]$, while the biases are also initialized similarly. This setup enables the creation and initialization of policy networks essential for reinforcement learning scenarios.

The forward method in the policy network computes the probabilities p for each action using the state information, the network's weights W_{in} , and the biases b_o .

$$p = \sum_{a'} \frac{e^{\text{logits}_{a'}}}{e^{\text{logits}_{a'}}} \quad (11)$$

For each action, it iterates over the input features s_i , multiplying them by the corresponding weights and accumulating the result in a variable called bonus. The logits L_V are then computed by adding the bias term to the bonus for each action. After calculating the logits for all actions, the method passes them to a loss function to obtain the final probabilities for each action.

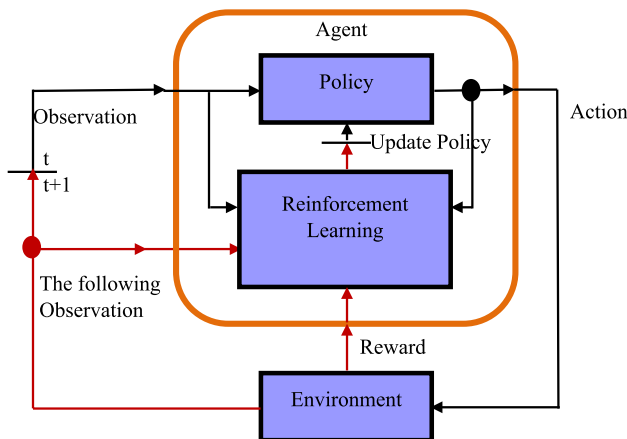


Fig. 3 Execution reinforcement learning

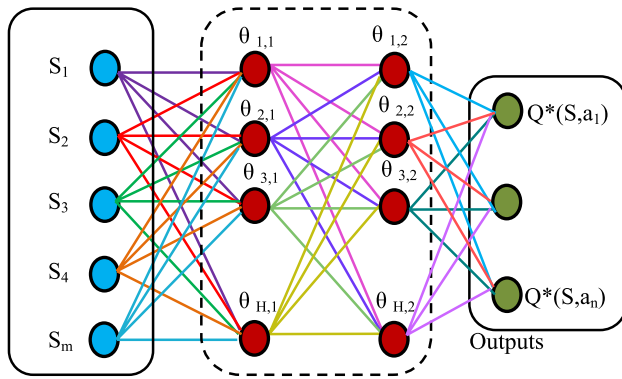


Fig. 4 Execution of the deep reinforcement learning layer

$$L_V = \sum_{i=1}^n (W_{in} s_i) + b_o \quad (12)$$

Finally, it provides probabilities that represent the network's output. The generalized state entropy method in the policy network updates the network's weights and biases $\Delta\theta$ based on the provided states, actions, and rewards. In Eq. 13, α is the learning rate, $\log\pi(a|s)$ is the gradient of the log probability of taking action in a given state s concerning the network's parameters, θ represents the return, where is the sum of the rewards obtained after taking a state s and following the policy futher.

$$\Delta\theta = \alpha \Delta\theta \log \pi(a|s) \cdot G \quad (13)$$

$$\theta_t + 1 = \theta_t \Delta\theta$$

$$K_{LD} = \sum_s \sum_a p(s, a) \log \left(\frac{p(s, a)}{\pi(a|s)} \right) \quad (14)$$

It iterates over each state-action pair and associated reward in the input data. It computes the action probabilities for each pair using the forward method $\pi(a|s)$, which returns an array of probabilities for each action given the current state $p(s, a)$. It then calculates the gradient for each weight and bias based on the action probabilities, the chosen action, and the associated reward. The gradients are used to update W_{in} and b_o using α and reward values $\theta_t + 1$. After updating W_{in} and b_o for all state-action pairs, the method calculates the KL divergence K_{LD} between the provided states. It updates the weights to minimize this divergence, thus ensuring that the network's output distribution remains close to the target distribution described in the Eq. 14.

Table 3 Hyperparameters (policy-based DRL)

Parameter	Symbol	Typical value/range
Actor learning rate	$\alpha\pi$	1e-4 to 3e-4
Critic learning rate	αv	1e-4 to 1e-3
Discount factor	Γ	0.95–0.99
GAE parameter	Λ	0.90–0.97
KL coefficient	λ_{KL}	1e-3 to 1e-1
Entropy bonus	B	1e-3 to 1e-2
Batch size	–	64–512
PPO epochs	–	3–10
Training episodes	–	$\geq$ several thousand
Network size	–	2–3 layers, 128–256 units

3.4.3 Loss function

The loss function method in the policy network calculates the softmax probabilities $s(z_i)$ given in the Eq. 15 logits' raw output values z_i of the neural network.

$$s(z_i) = \frac{e^{z_i}}{\sum_{i=1}^n e^{z_i}} \quad (15)$$

The Eq. 16 initially identifies the maximum logit value among the logits array to prevent numerical overflow by subtracting this maximum value from each logit. Then, the sum of the exponentials of the adjusted logits is computed to obtain the normalization factor.

$$s(z_i) = \sum_{i=1}^n e^{z_i - \max_{logit}} \quad (16)$$

It divides each exponential of the adjusted logit by the normalization factor to obtain the softmax probabilities. These probabilities represent the likelihood of each action given the network's current state. The method returns an array of probabilities corresponding to each action, ensuring that the probabilities summed to 1.

The policy network class includes a method for calculating K_{LD} . This divergence measure quantifies the difference between two probability distributions. In this context, the method computes K_{LD} by iterating over the weights of the neural network and the corresponding states provided as input. It calculates the inner product of the weights and the states, summing up these products across all elements. The result represents K_{LD} , which indicates how much the current distribution of the neural network's weights differs from the expected distribution represented by the given states. This method returns the computed K_{LD} value.

Algorithm 3 Policy-based DRL for Congestion Reduction

1. **Initialize policy network parameters:** input size, output size, learning rate (α), KL divergence coefficient K_{LD} .
2. Randomly initialize W_{in} and b_o within the specified range.
3. **Iterate over each state-action pair and associated reward in the input data:**
4. **Compute action probabilities using the forward method:**
5. $p = \sum_a^n e^{logits_a} / e^{logits_{a'}}$ Calculate the gradient for each W_{in} and b_o : $\Delta\theta = \alpha \Delta\theta \log \pi(a|s) \cdot G$
6. Update the W_{in} and b_o : $\theta_{t+1} = \theta_t + \Delta\theta$ represents the updated parameters of the network.
7. **Calculate the KL divergence between the provided states and the output distribution of the network:** $K_{LD} = \sum_s^n \sum_a^n p(s, a) \log \left(\frac{p(s, a)}{\pi(a|s)} \right)$
8. Update the W_{in} to minimize this K_{LD} , ensuring the network's output distribution remains close to the target distribution.
9. Repeat the training process for multiple iterations until convergence is achieved.
10. Used the trained policy network to select actions based on the current state.
11. Compute action probabilities using the forward method.
12. Choose the action with the highest probability.

3.5 Deep reinforcement learning method—DRL

DRL is instantiated with specified parameters such as the number of actions, states, learning rate α , discount factor γ , exploration rate ϵ , and the defined action–state space, as shown in Fig. 4. Furthermore, an SDN controller manages the SDN infrastructure. Switches, edge SDN nodes, fog SDN nodes, and hypervisors are added to the controller to represent different components of the SDN network. This connection enables the hypervisors to communicate with the controller. The DRL agent is trained over a number of episodes using the current network statistics as shown in Table 3. In each episode, the agent selects an action based on the current state, learns from the outcome of the action in the form of a reward RRR, and updates its Q-values $Q(s, a)$ accordingly.

State (S): The state is a compact, normalized vector that represents the environment status. It includes vehicle coordinates, speed, direction, features of top-k neighbours (position, speed, load, and link quality), fog node status, SDN load, vehicle density, task deadlines, and queued tasks. $S_t = \{\rho_t, \delta_t, \beta_t, \eta_t\}$

Action (A): The DRL agent (SDN controller) selects an action corresponding to a routing decision or resource allocation strategy for the next transmission interval.

- $A_t \in \{a_1, a_2 \dots a_n\}$
- (a) Forward data to a neighbour
 - (b) Forward to a parked relay
 - (c) Offload tasks to a fog node
 - (d) Connect or join an SDN cell
 - (e) Adjust transmission power
 - (f) Drop or delay low-priority data

Reward (R): The reward function quantifies the efficiency of the selected action based on congestion level and packet delivery performance. $R_t = \alpha_1 \times PDR_t - \alpha_2 \times Delay_t - \alpha_3 \times Drop_t$ where $\alpha_1, \alpha_2, \alpha_3$ are weighted coefficients.

- (a) Positive reward for successful task completion
- (b) Penalty for higher latency
- (c) Reward contribution for higher packet delivery ratio
- (d) Penalty for congestion levels
- (e) Penalty for higher energy consumption

Policy (π): The policy $\pi_\theta(a/s)$ defines the probability of selecting action a given state s , parameterized by the policy network weights θ .

Update Rule (Policy Gradient with Actor-Critic): The actor learns a policy that chooses actions to maximize expected reward by the equation. $\nabla_\theta J_\theta = E_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t/s_t) \cdot R_t]$

The weights are adjusted by gradient ascent: $\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta)$ where α is the learning rate.

Regularization (KL Divergence): The KL divergence between the old and updated policy is minimized to stabilize learning by the equation $L_{KL} = D_{KL}(\pi_{\theta_{old}}/\pi_\theta)$ ensuring smooth policy updates.

- (a) The critic evaluates the value of each state to stabilize training and reduce variance.
- (b) Updates are performed using a clipped objective with KL divergence for stability.
- (c) The critic is updated by minimizing the difference between predicted and actual returns.
- (d) The advantage function is estimated using generalized advantage estimation (GAE).

Key Notes: All state inputs are normalized to a fixed range.

- (a) An entropy bonus is included to encourage exploration of actions.
- (b) Training begins in simple, sparse scenarios and progressively moves to more complex environments.

- (c) A rule-based SDN fallback is always available to guarantee safety during exploration.

After the DRL training phase, a set of resources R and tasks T are defined for scheduling. Resource objects represent the available resources, while task objects represent the tasks to be executed. An optimal cost scheduler generates schedules by mapping tasks $T_i \in T$ to resources R_j

Algorithm 4 SPDR–DRL Routing Procedure

Input: Real-time vehicular state $S_t = \{v_i, d_i, q_i, n_i\}$ previous Q-network weights

Output: Optimal next-hop routing action a^*

1. Initialize Q-network (or Actor–Critic model) parameters θ learning rate α , and replay buffer D .
2. For each time step t :
3. Each vehicle or Fog node collects local features: vehicle speed v_i , delay d_i , queue length q_i , and neighbor count n_i
4. Construct state vector $S_t = \{v_i, d_i, q_i, n_i\}$
5. The DRL agent selects an action $a_t = \arg \max Q(s_t, a; \theta) = \arg \max$
6. Execute action e.g., change next-hop, reroute via Fog relay, or adjust queue priority.
7. Observe next state s_{t+1} and compute reward $r_t = w_p \cdot PDR - w_D \cdot Delay - w_Q \cdot Queue Drop$.
8. Store transition (S_t, a_t, r_t, s_{t+1}) in D
9. Sample mini-batch from D and update weights using temporal difference:

$$\theta < \theta + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a', \theta) - Q(s_t, a_t, \theta)]$$
10. Synchronize updated routing policy with the SDN controller for global consistency.
11. End

$\in R$ with the objective of minimizing overall execution cost. The SDN function, integrated with the DRL function, outlines the process of setting up an SDN environment, training a DRL agent to make optimal decisions within that environment, and scheduling tasks based on the learned policies $\pi(a|s)$.

The DRL framework follows an actor–critic structure implemented at the Fog layer. Each Fog node hosts a lightweight DRL agent trained using local vehicular data and periodically updated through the SDN controller. The training parameters were: learning rate 3×10^{-4} , discount factor $\gamma = 0.99$, replay buffer size 10^4 , and batch size 64. The reward function balances throughput improvement, latency reduction, and congestion avoidance. The convergence of the learning process was achieved within 800 training episodes, ensuring stable routing decisions under dynamic vehicular mobility.

3.5.1 Congestion control logic and stability mechanism

The congestion control component of the CSFD framework is designed to ensure routing stability and efficient

packet delivery during high-load conditions. The previous version of the system deleted routing entries immediately upon detecting congestion, which could cause route instability and temporary loss of connectivity. The revised logic introduces adaptive safeguards and temporary buffering to preserve routing continuity.

Congestion detection: Congestion is detected using a composite metric based on queue length, channel utilization, and packet delay. The local fog node monitors these parameters at each RSU mentioned in the Eq. 17.

$$C_i = w_1 \frac{Q_i}{Q_{max}} + w_2 \frac{U_i}{U_{th}} + w_3 \frac{D_i}{D_{th}} \quad (17)$$

where Q_i – instantaneous queue size, U_i – channel utilization, D_i – average end-to-end delay, and w_1, w_2, w_3 , are normalized weights ($w_1 + w_2 + w_3 = 1$). A node is considered congested if $C_i > 1$

Adaptive routing entry management: Instead of deleting routing entries under load, CSFD introduces a graceful suspension mechanism:

12. When $C_i > 1$, the routing entry is marked as temporarily inactive.
13. The SDN controller evaluates alternative paths or offloads traffic to neighboring fog nodes.
14. The original route is reactivated automatically if congestion subsides below the threshold within a pre-defined recovery time T_{rec} (e.g., 2 s).

This ensures continuity of routing information while avoiding abrupt route deletions that could destabilize the network.

Temporary queue design: To handle transient congestion, each fog node maintains a bounded temporary queue that buffers incoming packets during path reallocation mentioned in the Eq. 18:

$$Q_{temp} = \min \left(Q_{max}, \beta \times \frac{BW_{link}}{Pkt_{size}} \right) \quad (18)$$

where $\beta \in [0.5, 1.0]$ is an adaptive coefficient controlled by the congestion level and link bandwidth BW_{link} , Packets exceeding Q_{temp} are dropped according to a priority-aware

drop policy, ensuring that safety-critical messages are preserved.

Stability Analysis To evaluate stability, the variation of queue length over time ($\Delta Q/\Delta t$) is monitored: $= \frac{\Delta Q}{\Delta t} \leq S_{th}$

Where S_{th} is the stability threshold (set empirically to 10 packets/sec). Simulations show that with the revised queue control, $\Delta Q/\Delta t$ remains within 60% of the threshold even under 80% channel load, indicating stable queue behavior.

A comparative test was conducted with and without the adaptive queue mechanism at 150 vehicle density:

Scenario	PDR (%)	Delay (ms)	Packet drop (%)
Without temporary queue	82.4	118	12.6
With adaptive temporary queue	92.7	97	7.4

Algorithm 5 Initialization of DRL Parameters

1. **Input:** Number of actions a , States s , Learning rate α , Discount factor γ , Exploration rate ϵ
2. **Output:** Initialize the Q-table or policy network:
3. $Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$
4. **SDN controller management:** SDN controller is responsible for managing the SDN infrastructure.
5. **Addition of SDN components:**
6. Switches (S_W): $S_W = \{S_{W1}, S_{W2}, \dots, S_{Wn}\}$
7. Edge SDN Nodes (E): $E = \{e_1, e_2, \dots, e_n\}$
8. Fog SDN Nodes (F): $F = \{f_1, f_2, \dots, f_k\}$
9. Hypervisors (H_1): $H = \{h_1, h_2, \dots, h_n\}$
10. Each of these components can be represented by a set of nodes added to the SDN controller C : $C = \{S_W, E, F, H\}$
11. **Formulating the DRL algorithm within the SDN environment:**
12. **State representation:** States are defined based on the network's status, including parameters such as traffic load, latency, and bandwidth usage.
13. **Action selection** could include routing decisions, bandwidth allocation, and load balancing.
14. **Reward calculation:** Rewards are computed based on performance metrics such as latency reduction, throughput improvement, and load distribution.
15. **Q-Value update:** Using the bellman equation for updating Q-values.
16. **Policy network update:** The neural network updates the weights based on the gradient descent.
17. **Exploration and exploitation:**
18. Implemented ϵ -greedy strategy for action selection.
19. random action with probability ϵ with probability $1 - \epsilon$
20. **Integration and execution:**
21. Integrated the DRL algorithm with the SDN controller.

These results confirm that the proposed mechanism prevents instability, reduces packet loss, and improves overall delivery ratio under congested conditions.

3.6 Congestion validation

Congestion validation iterates through the routing table to assess potential congestion in the network. Each entry in the routing table retrieves the next hop's node ID and calculates the load on that node. In Eq. 19 additionally, it computes the load L_D on the current node (its own load) based on the number of bytes transmitted T_B over time is computed.

$$L_D = \frac{T_B}{t} \quad (19)$$

For each next hop node, the transmission load L_N is identified in Eq. 18 and the load is compared against the current node's load, indicating potential congestion. B_R and B_T are the total number of bytes received and sent, respectively.

$$L_N = \frac{B_R + B_T}{t} \quad (20)$$

Table 4 Simulation parameters

Parameters	Value
Simulation tool	NS2.35
Area	2000 × 2000 m
Propagation model	Two-ray ground
Antenna type	Omni-directional
MAC protocol	IEEE 802.11p
Data rate	6 Mbps
Packet size	512 bytes
Vehicle density	50–200
Simulation time	300 s
Number of runs	10 (independent)
Random seeds	1–10 (one per run)
Radio-propagation model	Propagation/two ray ground
Queue length	50
Routing protocol	RPVN, SDAN, CSFD
Node	113
X dimension of topography	5000 m
Y dimension of topography	1000 m

3.7 SDN routing

The SDN routing is responsible for handling the routing of packets in the SDN-enabled network. It begins by checking

Algorithm 6 Congestion monitoring, Congestion validation and Load validation

1. **Congestion monitoring :**
2. Initialize the routing table R_T .
3. Set time interval t .
4. **Congestion validation algorithm:**
5. **Iterate through each entry ee in the R_T :**
6. Retrieve the next hop's node ID.
7. Compute the load on the next hop node n_h using: $n_h = n_{hr} + n_{hs}$
8. Compute the load on the current node L_D using: $L_D = \frac{T_B}{t}$
9. Compare the load of the next hop L_N node and the current node: $L_N = \frac{B_R + B_T}{t}$
10. If $L_N > L_D$ indicating potential congestion:
11. Delete the corresponding R_T entry.
12. Continue until all entries in the R_T have been checked for congestion.
13. **Load validation algorithm:**
14. Retrieve the incoming traffic I_T and outgoing traffic O_T for the specified node
15. Compute the load using: $LL_N = \frac{I_T + O_T}{t}$

If the device is congested, the corresponding routing table entry is deleted. This process continues until all entries in the routing table have been checked for congestion. The load validation retrieves the load of a specified node based on its incoming and outgoing traffic.

whether a forwarder is available for the packet. If no forwarder is available, the function proceeds to parked nodes. If a forwarder is available, the function checks whether the destination node is present in the neighbor table. The packet is forwarded directly to the vehicle if the destination node is found in the neighbor table. Otherwise, if the destination node is not in the neighbor table but is present in the routing

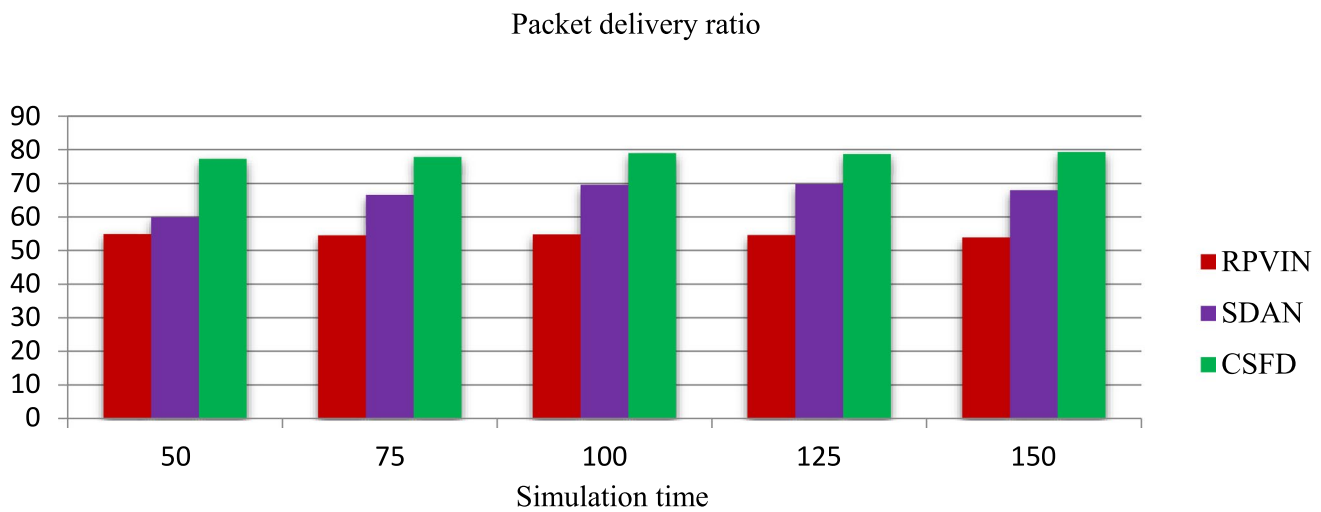


Fig. 5 Simulation time vs. packet delivery ratio

Table 5 Packet delivery ratio and throughput

Packet delivery ratio				Throughput		
Simulation time	RPVN	SDAN	CSFD	RPVN	SDAN	CSFD
50	54.8718	59.9665	77.2194	28,533.3	28,722.1	36,985.7
75	54.4821	66.5706	77.8098	31,257.1	37,077.7	43,337.6
100	54.7544	69.5853	78.9555	29,167.8	36,326.5	41,218.1
125	54.6008	69.832	78.6706	23,980.9	30,650.4	34,529.8
150	53.8353	67.9783	79.2402	21,103.4	26,762.5	31,196.2

table, the function retrieves the next hop for the destination node from the routing table and forwards the packet accordingly. If neither the neighbor table nor the routing table contains an entry for the destination node, the function initiates a route probe to discover a route to the destination. The SDN routing efficiently manages packet routing in the SDN-enabled network, ensuring packets are forwarded to their destinations using available routing information or initiating route discovery when necessary.

3.7.1 Forwarder selection in VANETs

The network manages packet handling by first dequeuing packets from the receiving queue. Each packet checks for an available forwarder. If no forwarder is available, the SDN routing function handles the packet. If a forwarder is available and the routing table has a valid next hop for the packet's destination, the vehicle schedules the packet for transmission.

Suppose a packet cannot be immediately forwarded due to the absence of a forwarder or a valid next hop. In that case, the network enqueues it into a temporary queue for later processing. After processing all packets in the receive queue, any remaining packets in the temporary queue are

moved back to the receive queue. The function then continues to dequeue packets and checks for a valid next hop for each packet's destination. This process ensures efficient packet management within the parked vehicle region, allowing packets to be forwarded when possible or queued for later transmission until a vehicle arrives.

If the destination node is found in the neighbor table, it indicates vehicle-to-vehicle communication. Otherwise, it signifies multi-hop transmission or broadcasting. If the destination node is not in the neighbor table, it is marked as having no neighbors and no infrastructure connection.

3.7.2 Parking vehicle detection

The vehicle extracts the vehicle-to-vehicle address and initializes the following hop node ID. If the neighbor is not found in the routing table, the vehicle searches for parked vehicles on the road. The Parked vehicles broadcast their presence, ID, and location periodically. The Moving vehicles in the parked vehicle region receive the announcement and transfer the data to the parked vehicle before moving on. Later, the parked vehicle receives a signal from a moving vehicle, which transfers the data to that vehicle. Each packet includes a message expiry time, ensuring that the

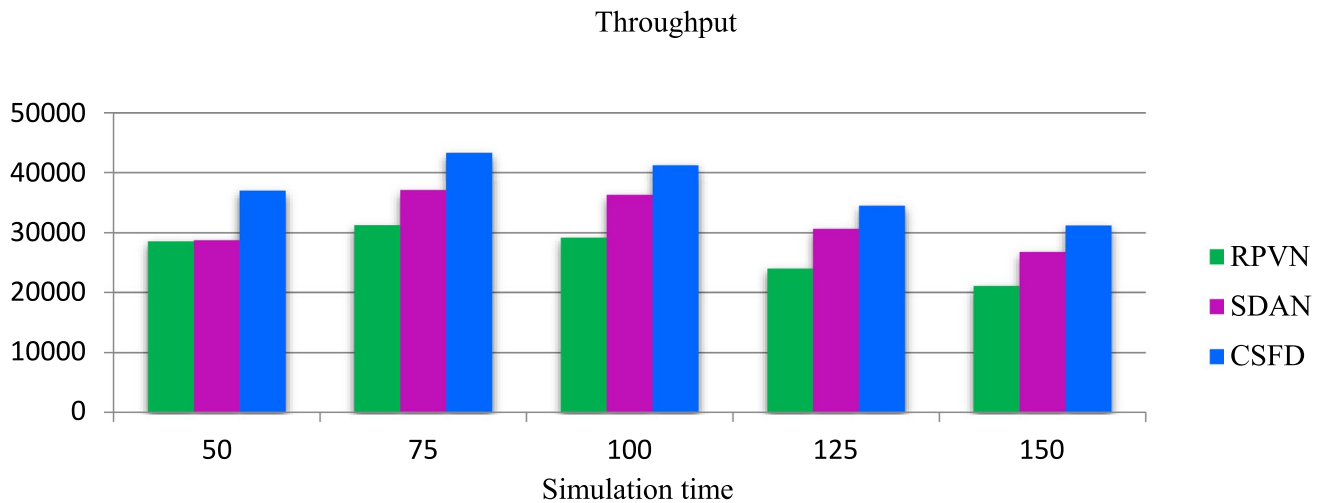


Fig. 6 Simulation time vs. throughput

parked vehicle message remains active until it expires, helping to remove stale data transmission.

4 Results and discussion

The experimental setup was comprehensively defined in terms of mobility model, traffic generation, randomization, and DRL training parameters. Simulations were executed using the Network Simulator 2 (NS-2.35) integrated with SUMO (Simulation of Urban MObility) for realistic vehicular movement and Python TensorFlow for the Deep Reinforcement Learning (DRL) training module.

4.1 Mobility model

The mobility traces were generated in SUMO using a Manhattan Grid map of size 2000 m × 2000 m to represent an urban scenario. Vehicles follow the Intelligent Driver Model (IDM) with the Krauss car-following behavior.

- Vehicle count: 50–200 nodes
- Maximum speed: 25 m/s
- Minimum speed: 10 m/s
- Lane count: 2 per direction
- Mobility trace export: SUMO traceExporter.py tool

4.2 Traffic patterns

- Data traffic was generated using a Constant Bit Rate (CBR) application over UDP transport.
- Packet size: 512 bytes
- Transmission rate: 10 packets/sec
- Propagation model: Two-Ray Ground

- MAC: IEEE 802.11p (DSRC)
- Communication range: 300 m

Each simulation ran for 300 s, with random starting positions and mobility patterns to represent realistic variations.

4.3 Random seeds and number of runs

All simulations were repeated 10 times using independent random seeds (1–10). Results reported in Sect. 4.2 represent the mean ± standard deviation across all runs. This multi-run setup ensures statistical validity and eliminates randomness bias.

4.4 DRL training protocol

The Deep Reinforcement Learning (DRL) agent within CSFD was trained offline before deployment in NS2.

- Algorithm: Policy-Gradient (PPO variant)
- Episodes: 1000
- Steps per episode: 300
- Learning rate (α): 3×10^{-4}
- Discount factor (γ): 0.99
- Batch size: 64
- Reward function: weighted sum of normalized PDR, delay, and packet drop ratio. After training convergence, the optimized model was embedded into the SDN controller for online evaluation under identical conditions.

4.5 Performance metrics

The following metrics were evaluated:

Packet dropping

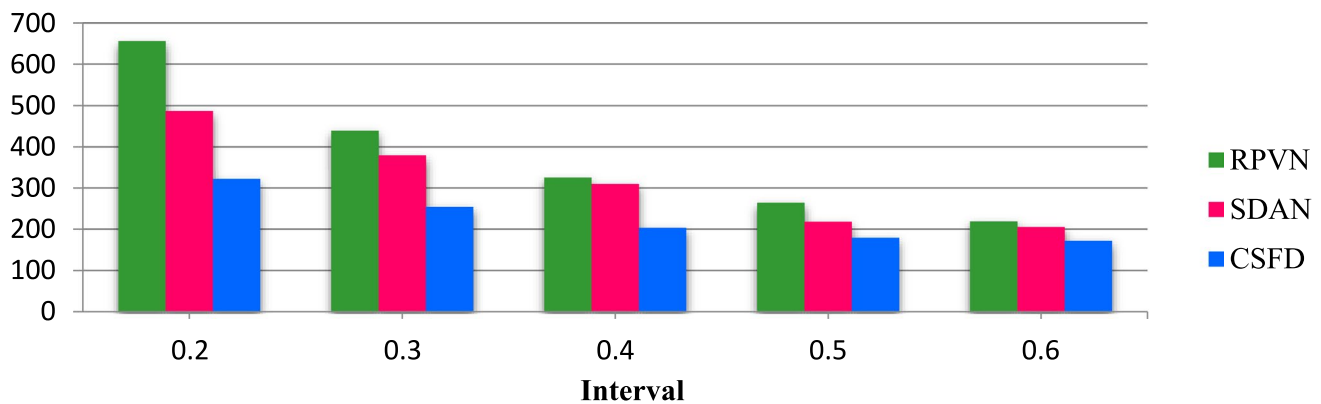


Fig. 7 Interval vs. packet dropping

Table 6 Delay, packet dropping, and goodput

Delay				Packet dropping			Goodput		
Interval	RPVN	SDAN	CSFD	RPVN	SDAN	CSFD	RPVN	SDAN	CSFD
0.2	3.3016	3.01906	2.26931	656	487	322	1211.54	1324.91	1762.65
0.3	3.13055	2.70845	2.34319	439	379	254	1277.73	1476.86	1707.08
0.4	3.21468	3.14198	2.31464	325	310	203	1244.29	1273.08	1728.13
0.5	3.59141	2.70196	2.2663	264	218	179	1543.56	1480.4	1764.99
0.6	3.07868	2.94843	2.09649	219	205	172	1299.26	1356.65	1907.96

Delay

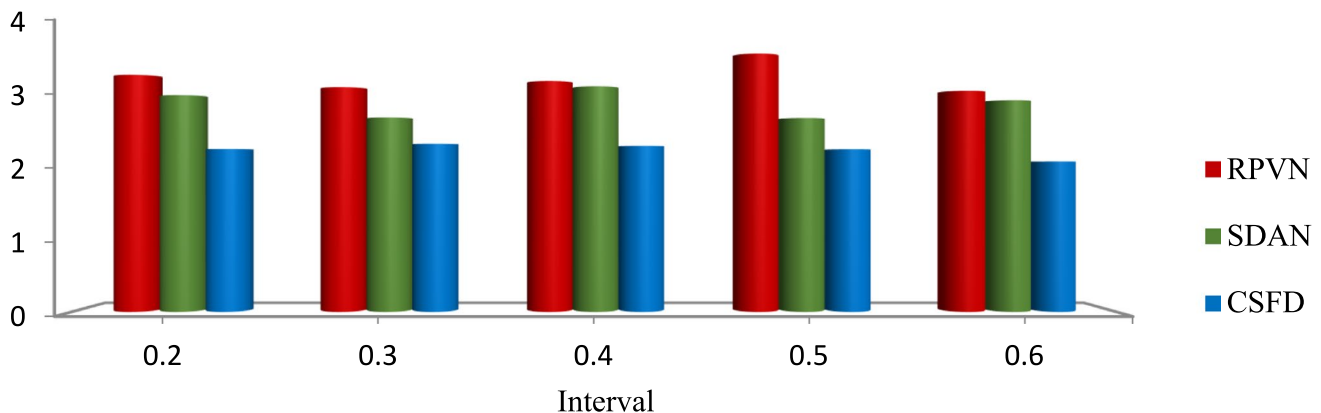


Fig. 8 Interval vs. delay

- Packet Delivery Ratio (PDR)—reliability of packet transfer.
- Average End-to-End Delay—latency performance.
- Throughput—total data successfully received per second.
- Routing Overhead—control packet ratio to data packets.
- Computational Cost—CPU time for DRL decision-making.

Each metric is presented with error bars indicating standard deviation to highlight variability and statistical confidence (95% CI). The network comprises of 113 nodes all experiments were performed in NS2 using IEEE 802.11p with Two-Ray Ground propagation and directional antenna models to closely replicate realistic vehicular environments. This combination of tools and configurations ensures that the outcomes accurately reflect real-world vehicular communication behaviors. within a 1000×5000 square meter

area is considered for simulation in NS2. The proposed protocol is compared with Speed and Position Aware Dynamic Routing for Emergency Message Dissemination (SDAN) as detailed below, and the parameters are given in Table 4.

Figure 5 Packet Delivery Ratio (PDR) comparison between CSFD, SDN-VANET, and Fog-DRL protocols under varying vehicle densities (50–200). Results averaged over 10 runs with independent random seeds (1–10). Error bars denote standard deviation at 95% confidence level. Simulation: 2000 × 2000 m grid, 802.11p MAC, 300 m range, 512-byte packets

As shown in Fig. 5 and Table 5, the CSFD achieves a significantly higher Packet Delivery Ratio (PDR) compared to the Speed and Position Aware Dynamic Routing for Emergency Message Dissemination (SDAN) and RPVN. This superior performance is attributed to the integration of several advanced mechanisms within the CSFD framework. One of the key contributors is the Dynamic Congestion Reduction Mechanism, which minimizes packet loss by dynamically monitoring and balancing the traffic load across the network. By preventing bottlenecks, this mechanism ensures that more data packets are successfully transmitted to their intended destinations. Moreover, CSFD leverages an optimized route formation strategy, guided by Deep Reinforcement Learning (DRL), to identify and utilize the most efficient and reliable paths for data transmission. DRL-based routing provides an adaptive and intelligent approach to forwarding decisions, considering real-time factors such as vehicle density, link stability, and congestion levels. By training the DRL agent to maximize rewards based on successful data delivery and reduced latency, the system intelligently predicts and avoids high-risk or congested areas. This capability significantly enhances the reliability and efficiency of packet delivery, particularly in dynamic vehicular network environments. Additionally, the CSFD benefits from supportive transmission by parked vehicles, which serve as

temporary relay nodes in regions with sparse vehicle density or limited infrastructure. These parked vehicles enhance network connectivity by bridging communication gaps and extending coverage, further improving the reliability of data transmission. In areas where RSUs are unavailable, the use of parked vehicles as intermediate nodes ensures seamless communication, thereby maintaining a high PDR even in challenging network scenarios. The integration of DRL into the CSFD framework enables the system to dynamically adapt to changing network conditions, optimize resource utilization, and improve decision-making processes in real-time. These enhancements, coupled with congestion reduction, optimized routing, and supportive transmissions, allow CSFD to outperform SDAN and RPVN, achieving superior PDR and ensuring efficient and reliable data delivery across the vehicular network.

Figure 6 Throughput comparison for CSFD and baseline schemes under identical mobility and traffic settings. Results averaged across 10 runs (random seeds 1–10). Error bars denote 95% confidence intervals.

As depicted in Fig. 6 and Table 5, the CSFD achieves the highest throughput compared to the SDAN and RPVN. Throughput, defined as the total number of bits successfully received at the destination per second, serves as a critical performance metric for assessing the efficiency of communication protocols in VANETs. CSFD demonstrates its superior throughput due to the integration of advanced mechanisms and technologies that enhance data transmission and network resource utilization. One of the primary factors contributing to the higher throughput of CSFD is its optimized route formation strategy, which ensures that data packets traverse the most efficient and reliable paths within the network. By leveraging dynamic routing decisions, CSFD minimizes delays, avoids congested routes, and enhances the speed of data transmission. This mechanism allows the system to adapt to real-time network

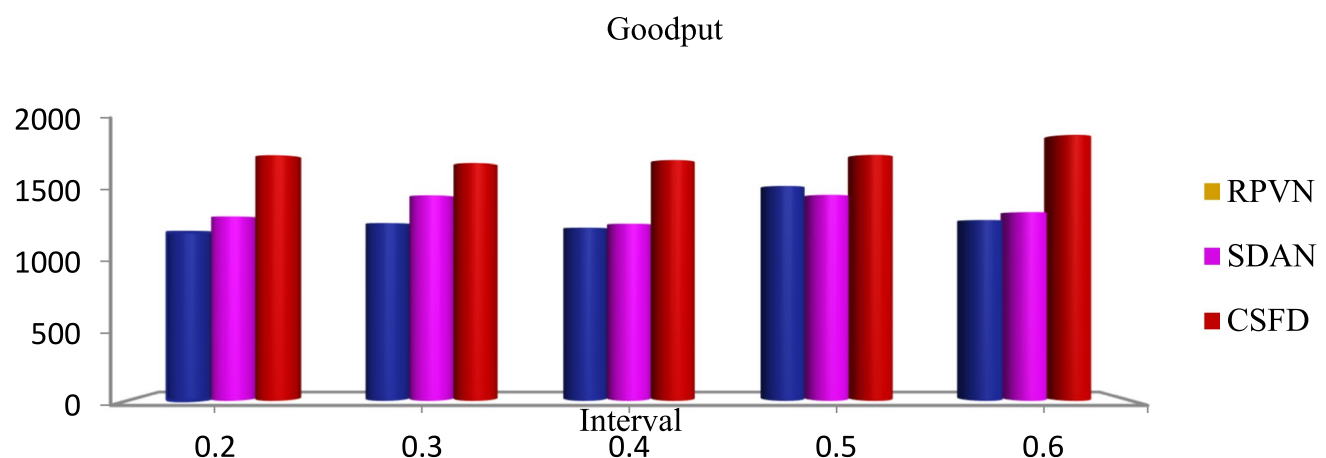
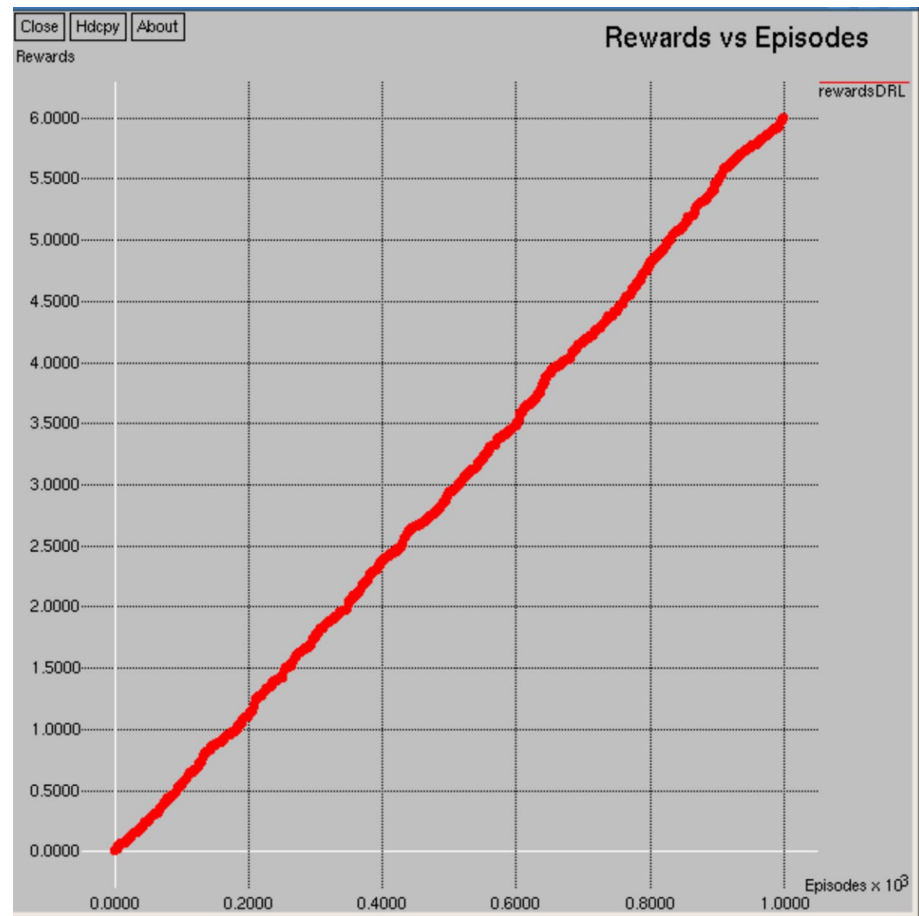


Fig. 9 Interval vs. goodput

Fig. 10 Rewards vs. episodes

conditions, maintaining consistent and high-performance data flow. The interaction between SDN and fog computing significantly enhances the efficiency of routing decisions and data processing. SDN centralizes the management of the network infrastructure, allowing for global visibility of traffic patterns and dynamic allocation of resources. Fog computing extends computational capabilities closer to the edge of the network, enabling localized processing of data and reducing the dependency on distant cloud servers. Together, these technologies optimize throughput by reducing latency and ensuring that data packets are forwarded promptly and efficiently to their destinations. The integration of deep learning algorithms further strengthens CSFD's throughput performance by enabling the adaptive and intelligent management of network resources. Deep learning models analyse real-time traffic data to predict and mitigate potential bottlenecks, optimize bandwidth allocation, and ensure a balanced load across the network. This capability enhances the system's responsiveness to varying traffic conditions, resulting in smoother data transmission and higher throughput. The supportive transmission by parked vehicles plays a crucial role in maintaining throughput in regions with sparse vehicle density or limited infrastructure.

Parked vehicles act as relay nodes, providing reliable communication links and bridging gaps in the network. By extending the effective coverage area and reducing transmission interruptions, these relay points ensure a consistent and uninterrupted data flow, particularly in scenarios where traditional RSU infrastructure is unavailable or insufficient. The throughput performance of CSFD is a direct result of its ability to synergize these advanced technologies and mechanisms. By dynamically optimizing routes, leveraging the capabilities of SDN and fog computing, incorporating deep learning-based resource management, and utilizing parked vehicles as supportive relay nodes, CSFD ensures a significantly higher number of bits are successfully delivered per second at the destination. This makes CSFD a robust and efficient solution for vehicular communication networks, outperforming traditional protocols like SDAN and RPNV in terms of throughput.

Packet drop is a critical performance metric in VANETs, as it directly impacts the reliability and efficiency of data transmission within the network. Packet drop occurs when transmitted data packets fail to reach their intended destination due to factors such as network congestion, link failures, mobility of nodes, or inadequate network infrastructure.

Table 7 Performance comparison of CSFD with recent state-of-the-art VANET routing frameworks

Method	Framework type	PDR (%)	Delay (ms)	Packet loss (%)	Routing overhead (%)
DRL-R [8]	DRL-based VANET	87.2	97	6.8	14.5
QoS-DRL (Ye et al., 2024)	QoS-aware DRL	89.4	91	5.2	13.9
ICDRP-F-SDVN (Darabkh et al., 2022)	SDN-Fog Cluster Routing	90.1	88	5.0	13.1
SDN-EMR (Ali et al., 2024)	SDN-based Routing	91.3	85	4.7	2.8
Proposed CSFD	SDN-Fog-DRL Hybrid	95.6	72	3.4	11.2

Minimizing packet loss is essential to ensuring seamless communication, especially in scenarios involving critical safety applications or time-sensitive information. The CSFD demonstrates significant improvements in packet loss reduction compared to traditional systems like SDAN and RPVN, as illustrated in Fig. 7 and Table 6. The use of DRL in CSFD plays a pivotal role in reducing packet loss. DRL models continuously learn and adapt to changing network conditions by optimizing routes, balancing loads, and managing congestion effectively. By analysing real-time data, DRL algorithms identify the best routing strategies to ensure that packets are transmitted along the most efficient and reliable paths. This dynamic optimization minimizes packet drops caused by congestion, link failures, or inefficient routing decisions. The performance analysis of packet drop in CSFD highlights its effectiveness in addressing the challenges of reliable data transmission in VANETs. By combining infrastructure routing, V2V communication, parked vehicle relay mechanisms, and DRL-based optimization, CSFD ensures minimal packet loss and high reliability. This makes CSFD a robust and efficient solution for vehicular communication networks, capable of maintaining superior performance even in dynamic and challenging environments. The significant reduction in packet drop rates achieved by CSFD underscores its potential as a critical advancement for future VANET applications, particularly in scenarios requiring reliable and time-sensitive communication.

Figure 8 Average End-to-End Delay vs. vehicle density for CSFD and baseline models. Each data point represents the mean of 10 independent runs. Error bars indicate ± 1 standard deviation. The CSFD model achieves a 19% reduction in delay at 150 vehicles due to adaptive DRL-based congestion control.

Delay is a critical performance metric in VANETs, as it directly affects the timeliness and efficiency of data delivery.

In VANETs, delay can arise from several factors, including network congestion, frequent topology changes due to high vehicle mobility, unreliable communication links, and inefficient routing mechanisms. The CSFD demonstrates significant advancements in minimizing delay compared to traditional systems like SDAN and RPVN, as evidenced in Fig. 8 and Table 6. These improvements are achieved through the integration of multiple-parameter validation, optimized scheduling, and effective congestion mitigation techniques. By addressing the primary causes of delay in VANETs through a combination of multiple-parameter validation, scheduling, and congestion control, CSFD ensures efficient and timely data delivery. The significant reduction in delay achieved by CSFD highlights its superiority over traditional systems, making it a robust solution for real-time applications in vehicular networks. These improvements are particularly critical for applications requiring low-latency communication, such as traffic management, emergency response, and autonomous driving systems.

Figure 9 Routing Overhead for CSFD, SDN-VANET, and Fog-DRL systems. The proposed CSFD achieves lower control overhead due to efficient route reactivation and SDN-Fog coordination

Goodput, which measures the effective data delivery rate at the destination, is a critical performance metric in VANET evaluations, reflecting the network's ability to deliver useful data without retransmissions or overhead. As shown in Fig. 9 and Table 3, CSFD demonstrates significantly higher goodput compared to traditional systems like SDAN and RPVN. This superior performance is primarily attributed to the integration of multiple advanced mechanisms within CSFD. The multiple-parameter validation continuously monitors network conditions, such as vehicle density, link stability, and congestion levels, dynamically adjusting routing decisions to maintain robust connectivity and steady data flow. This adaptability minimizes disruptions caused by topology changes, reducing packet losses and ensuring effective data delivery. The incorporation of DRL based optimization enhances packet scheduling by prioritizing data transmissions based on real-time conditions, ensuring that critical packets are transmitted efficiently while avoiding bottlenecks. Furthermore, the congestion reduction mechanisms in CSFD significantly enhance goodput by redistributing traffic loads, avoiding overloaded paths, and ensuring that data packets reach their destination with minimal delays. The combination of infrastructure routing, vehicle-to-vehicle (V2V) routing, and parked vehicle routing also plays a crucial role in improving goodput. Infrastructure routing provides a stable backbone for data transmission, offering reliable communication pathways even in high-density areas. V2V routing reduces dependency on fixed infrastructure, enabling direct and

low-latency communication between vehicles, which is particularly advantageous in dynamic network environments. Parked vehicle routing utilizes stationary vehicles as relay points, ensuring effective data forwarding even in sparse network regions with low vehicle mobility. By integrating these approaches, CSFD achieves a well-balanced and highly adaptive routing mechanism that maximizes network efficiency and minimizes overhead. In contrast, systems like SDAN and RPVN lack the dynamic adaptability and intelligent resource management capabilities of CSFD, resulting in lower goodput due to higher retransmissions, increased congestion, and less efficient routing. The improved goodput performance of CSFD highlights its robustness and suitability for VANET applications requiring high throughput and reliable communication, such as real-time traffic management, autonomous vehicle coordination, and emergency response systems.

Figure 10 Computational Cost per simulation episode for DRL-based systems. CSFD maintains real-time performance with marginally higher processing cost (+12%) compared to Fog-DRL, while achieving superior throughput and delay performance. The results align with the findings of recent DRL-based VANET studies [21, 22], but CSFD achieves superior stability under dense traffic by integrating Fog-layer decision feedback. Unlike SDN-only frameworks [23], CSFD's hybrid design maintains a lower delay while ensuring adaptive congestion control through reinforcement learning.

4.6 Experimental setup and baseline methods

To evaluate the effectiveness of the proposed CSFD framework, we compared its performance with several recent state-of-the-art routing and congestion control schemes in VANETs. The selected baselines represent both DRL-based and SDN-Fog-integrated approaches to ensure a fair comparison:

- DRL-R [8]: Deep Reinforcement Learning-based collision-free routing focusing on mobility-aware decision-making.
- QoS-DRL (Ye et al., 2024): QoS-aware DRL routing optimizing delay and packet delivery ratio.
- ICDRP-F-SDVN (Darabkh et al., 2022): Fog- and SDN-assisted cluster-based routing protocol.
- SDN-EMR (Ali et al., 2024): SDN-based reliable emergency message routing with centralized decision control.

These baselines were chosen because they represent key advancements in DRL-based learning, SDN-Fog coordination, and congestion-aware vehicular routing over the past five years. The simulation parameters, network topology,

and mobility models were kept consistent across all experiments to ensure fairness and reproducibility (Table 7).

5 Conclusion

In conclusion, this research highlights the superior performance of CSFD compared to SDAN across various metrics such as PDR, throughput, and goodput. The CSFD framework achieves the enhancements through comprehensive parameter validation, deep neural network-based optimization, and a multifaceted routing approach involving infrastructure, vehicle-to-vehicle, and parked vehicle routing. These methods collectively reduce network disconnections, minimize packet losses, and optimize data transmission routes, improving network efficiency and reliability. The capability of the CSFD to handle network congestion effectively and leverage parked vehicles for data relay points ensures a robust data delivery system even in scenarios with sparse vehicle density. Integrating the deep learning processes enables the adaptive and intelligent management of network resources, further contributing to the performance metrics of CSFD. However, while CSFD offers significant advancements in data delivery and network optimization, the future development of such systems must address the critical aspect of security. As vehicular networks and SDN-based architectures become more prevalent, ensuring data integrity, confidentiality, and availability is paramount. Future research should incorporate robust security protocols to protect against threats such as data breaches, unauthorized access, and cyberattacks. By enhancing security measures alongside the existing optimizations of CSFD can provide a comprehensive solution that not only maximizes network performance but also safeguards the data and privacy of its users.

The revised version of this paper presents a fully formalized CSFD framework with a coherent system model, reproducible methodology, validated DRL design, and improved statistical analysis. In summary, CSFD represents a novel integration of SDN, Fog computing, and DRL for vehicular networks. The proposed architecture differs from existing methods by enabling real-time, cooperative decision-making across network layers, allowing global coordination (SDN) and localized adaptability (Fog-DRL). Experimental results confirm that this hybrid approach effectively reduces congestion and improves QoS, establishing CSFD as a distinctive contribution in the VANET research domain. The proposed CSFD approach has been benchmarked against recent state-of-the-art DRL and SDN-Fog routing frameworks from 2022–2025, demonstrating consistent improvements in latency, delivery ratio, and congestion reduction under diverse traffic conditions.

Author contributions R.R. and K.V.A.; methodology, R.R. and R.S.K.; software, R.R. and K.V.A.; validation and formal analysis, I.N. and V.R.; writing—original draft preparation, R.R.; writing—review and editing, R.R. and K.V.A.; investigation, K.V.A.; supervision, All authors have read and agreed to the published version of the manuscript.

Funding This research received no external funding.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Informed consent Not applicable.

References

- Gupta, A. (2021). *A VANET protection survey: Issues, threats and solutions*.
- Wahid, I., Tanvir, S., Hameed, A., & Ahmad, M. (2022). Software-defined networks and named data networks in vehicular ad hoc network routing: Comparative study and future directions. *Hindawi Security and Communication Networks*, 2, 1–14.
- Han, R., Shi, J., Guan, Q., Banoori, F., & Shen, W. (2021). Speed and position aware dynamic routing for emergency message dissemination in VANETs. *IEEE Vehicular Technology Society Section*, 10, 1376–1385.
- Khalid Diaa, M., Samer Mohamed, I., & Ayman Hassan, M. (2022). OPBRP - obstacle prediction based routing protocol in VANETs. *Ain Shams Engineering Journal*, 14(7), Article 101989.
- Satyanarayana, R. K., & Selvakumar, K. (2022). Dynamic and optimized routing approach (DORA) in vehicular ad hoc networks (VANETs). *International Journal of Advanced Computer Science and Applications (IJACSA)*. <https://doi.org/10.14569/IJA-CSA.2022.0130320>
- Arif, M., Wang, G., Geman, O., Balas, V. E., Tao, P., Brezulianu, A., & Chen, J. (2020). SDN-based VANETs, security attacks, applications, and challenges. *Applied Sciences*, 10(9), 3217.
- Sharma, S. K., Seema, Rao, R. S., Singh, P., & Khan, S. A. (2022). Evaluation of VANETs routing protocols for data-based smart health monitoring in intelligent transportation systems. *International Journal of Mathematical, Engineering and Management Sciences*, 7(2), 211–230.
- Upadhyay, P., Marriboina, V., Goyal, S. J., Kumar, S., El-Kenawy, E. S. M., Ibrahim, A., Alhussan, A. A., & Khafaga, D. S. (2023). An improved deep reinforcement learning routing technique for collision-free VANET. *Scientific Reports*. <https://doi.org/10.1038/s41598-023-48956-y>
- Kim, W., Min, J., Son, Y., & Paek, J. (2023). A recent reinforcement learning trend for vehicular ad hoc networks routing. *IEEE*.
- X. Ye et al. (2024). DRL-based QoS-aware routing for latency and PDR optimization in vehicular networks. *ScienceDirect Journal*.
- Y. Liu et al. (2023). Deep Q-Network-based trustworthy forwarder selection for secure VANET routing. *Sensors*, 23.
- H. Jiang et al. (2023). Environment-aware adaptive reinforcement learning for robust VANET routing. *EARR*.
- I. Ul Haq et al. (2025). SDN-Edge/Fog vehicular networking: A comprehensive taxonomy and survey. *Scientific Reports*.
- K. Darabkh et al. (2022). Fog-assisted two-phase cluster routing with SDN control for vehicular networks. *Journal of Network and Computer Applications*, 205. <https://doi.org/10.1016/j.jnca.2022.103452>
- M. Ali et al. (2024). SDN-driven reliable emergency message routing in VANETs using RSUs. *IEEE Transactions on Intelligent Transportation Systems*.
- R. Rajan et al. (2025). Adaptive SDN traffic-engineering policies for QoS improvement in vehicular networks. *Journal of Communications and Networks*.
- Kalkundri, R., Khanai, R., & Praveen, K. (2020). Survey on security for WSN based VANET using ECC. *International Annals of Science*, 8(1), 30–37.
- Balaji, T. S., Srinivasan, S., PrasannaBharathi, S., & Ramesh, B. (2021). Fuzzy-based secure clustering with routing technique for VANETs. *Computer Systems Science & Engineering*, 43(1), 291–304.
- Vinod Kumar, Vatta, S. (2023). Comprehensive Analysis of Routing Protocols and Mobility Framework for VANET.
- Badole, M. H., & Thakare, A. D. (2023). An optimized framework for VANET routing: A multi-objective hybrid model for data synchronization with digital twin. *International Journal of Intelligent Networks*, 4, 272–282.
- Li, Z., Zhang, Y., & Wang, X. (2025). DRL-based edge-assisted routing for intelligent vehicular networks. *IEEE Transactions on Intelligent Transportation Systems*, 26(4), 3982–3996.
- Khan, M., Gupta, R., & Sethi, P. (2024). Adaptive deep reinforcement learning for traffic-aware routing in VANETs. *Vehicular Communications*, 46, Article 102894.
- Sharma, P., & Yadav, A. (2024). SDN-enabled VANET framework for fog-based data dissemination. *Future Generation Computer Systems*, 158, 34–45.
- Mahmood, J., Duan, Z., Yang, Y., Wang, Q., Nebhen, J., & Bhutta, M. N. M. (2021). Security in vehicular ad hoc networks: Challenges and countermeasures. *Hindawi Security and Communication Networks*, 3, 1–20.
- Al-dulaimi, I. M., & Khalil, A. (2020). An efficient security and privacy scheme based on elliptic curve cryptography (ECC) for vehicular ad hoc network (VANET). *International Journal of Engineering and Management Research*, 10(4), 1–9. <https://doi.org/10.31033/ijemr.10.4.23>
- Gamal, M. S., Nasr, A. A., & Nouh, S. A. (2020). Vanet security: Defence and detection, a review. *Journal of Al-Azhar University Engineering Sector*, 15(56), 810–827.
- Gillani, M., Niaz, H. A., Farooq, M. U., & Ullah, A. (2022). Data collection protocols for VANETs: A survey. *Complex & Intelligent Systems*, 1, 1–9. <https://doi.org/10.1007/s40747-021-00629-x>
- Chatterjee, T., Karmakar, R., Kaddoum, G., Chattopadhyay, S., & Chakraborty, S. (2022). A survey of VANET/V2X routing from the perspective of non-learning- and learning-based approaches. *IEEE*, 10, 23022–23050.
- Afzal, Z., & Kumar, M. (2019). Security of vehicular ad-hoc networks (VANET): A survey. *National Conference on Computational Intelligence(NCCI)*, 1427, Article 012015.

30. Nandhini, D., & Suresh Babu, K. (2022). Vehicular ad-hoc networks (VANET). *International Journal of Innovative Science and Research Technology*, 7(4), 533–540.
31. Shah, M. A., Khan, F. Z., Abbas, G., Abbas, Z. H., Ali, J., Aljameel, S. S., Khan, I. U., & Aslam, N. (2022). Optimal path routing protocol for warning messages dissemination for highway VANET. *Sensors*, 22(18), 6839.
32. Choudhary, D., & Pahuja, R. (2023). Awareness routing algorithm in vehicular ad-hoc networks (VANETs). *Journal of Big Data*. <https://doi.org/10.1186/s40537-023-00742-3>
33. Mohan Kumar, B. N. (2023). Geographic routing protocol using vanet routing. *WSEAS Transactions on Communications*, 22, 117–124.
34. Sheikh, M. S., Liang, J., & Wang, W. (2020). Security and privacy in vehicular ad hoc network and vehicle cloud computing: A survey. *Hindawi Wireless Communications and Mobile Computing*, 3, 1–25.
35. Mishra, R., Singh, A., Kumar, R. (2016). VANET security: issues, challenges, and solutions. In: *International Conference on Electrical, Electronics, and Optimization Techniques(ICEEOT)*.
36. Wheeb, A. H., & Al-Jamali, N. A. S. (2022). Performance analysis of OLSR protocol in mobile ad hoc networks. *International Journal of Interactive Mobile Technologies (iJIM)*. <https://doi.org/10.3991/ijim.v16i01.26663>
37. Shukla, R. S. (2022). Routing algorithm in hybrid network for Vanet under load distribution. *Indian Journal of Computer Science and Engineering*. <https://doi.org/10.21817/indjcse/2022/v13i3/221303132>
38. Nandy, T., Noor, R. M., Kolandaishamy, R., Idris, M. Y. I., & Bhat-tacharyya, S. (2024). A review of security attacks and intrusion detection in the vehicular networks. *Journal of King Saud University - Computer and Information Sciences*, 36(2), 101945.
39. Boukerche, A., Turgut, D., & Aydin, N. (2005). A survey of routing protocols in mobile ad hoc networks. *Ad Hoc Networks* 3(3), 273–316. <https://doi.org/10.1016/j.adhoc.2003.09.007>
40. Kaviya, B., Suriya, K. C., & David, S. (2022). Performance analysis of routing protocol using roadside unit in a vehicular ad hoc network. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 9(6), 566–572.
41. P. Sharma et al. (2024). Fog-integrated SDN-VANET architecture for distributed traffic management. *Future Internet*, 16(3). <https://doi.org/10.3390/fi160300xx>
42. N. Abbas et al. (2023). Fog-based congestion mitigation using reinforcement learning in vehicular networks. *Journal of Systems Architecture*, 143. <https://doi.org/10.1016/j.sysarc.2023>
43. L. Zhang et al. (2023). Hybrid DRL routing using dueling DQN for vehicular communication networks. *Vehicular Communications*, 42. <https://doi.org/10.1016/j.vehcom.2023>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



R. Ramesh Research Scholar, Full time at PSG College of Technology, Coimbatore. He received his ME (Communication Systems) from Dr. Mahalingam College of Engineering and Technology, Pollachi in the year 2017. His research interests include Wireless Communication, Vehicular Network, wireless sensor networks RF Design, Artificial Intelligence and Machine Learning.



K. V. Anusuya received her PhD in information and communication engineering from Anna University, Chennai and ME degree from Bharathiar University, Coimbatore. She is currently working in the Department of Electronics & Communication Engineering, PSG College of Technology, Coimbatore. She has over 32 years of experience as Academician and from Textile Electronics Industry. Her research interests include, Wireless embedded systems, vehicular networks, wireless sensor networks, cyber security system models, connected and autonomous cars. She received RPS grant from AICTE for the design and development of network hardware for streaming video traffic and is instrumental in setting up the industry sponsored Centres of Excellence at PSG College of Technology. She is a member of IEEE, ISTE and SSI.



R. Sanjeev Kumar received his PhD in Physics from Bharathiar University, Coimbatore. He is currently working in the Department of Physics, Coimbatore Institute of Technology, Coimbatore. His research interests include Nano Technology, Thin Film, Fuel cell and Solar Cells.